FigurAI

By Andrew Moffitt

Contents

Analysis **2**
Description of project 2
UML Diagram 3
Requirements Specification 3
Project Plan 5

Design **7**
GUI Design 7
Project Design 15

Implementation **24**
Evidence 24

Testing **30**
End-user requirement testing 30
Functional requirement testing 31
Functional requirement testing evidence 32

Evaluation **40**
Fitness for purpose 40
Robustness 40
Maintainability 40

Appendix **41**
Key 41
FigurAI.py 41
init.py 55
bitmap.py 58
dbConn.py 61
shapeGen.py 63

Analysis

Description of project:

My project is an artificial intelligence program designed to train a digital neural network to correctly distinguish between a Circle, Triangle, Rectangle, Pentagon and a Hexagon. At the start of the program, the player is given a menu screen where they can choose to view information about the program, change program settings, train the AI, test the AI or exit the program. If the user chooses to train the AI, the program will randomly generate shapes, and for each shape the AI will guess what shape was generated by comparing it to the "weights" of each different kind of shape. If the program guesses incorrectly, the weights of the shapes will be adjusted slightly to make them more accurate. If the user chooses to test the AI, they can draw a shape on the screen using the mouse and the program will use the weights that were created by training the AI to guess which shape the user selected. I plan on using Python and the Pygame framework for the coding, and Google Colab for the majority of the AI training.

How it meets the Criteria:

My project is predominantly software development but integrates with database development.

The software development criteria satisfied are as follows:

- The project is object oriented because the different kinds of shape and all the buttons on the screen will be defined using classes
- The program will store a shape object for each kind of shape the program can generate in an array of objects
- A sorting algorithm will be used to sort the results of testing the AI in order from highest to lowest percentage certainty, and in training mode it will be used to sort the individual accuracies of the shapes in order from highest to lowest accuracy.

The database development criteria satisfied for the integration aspect of my project are as follows:

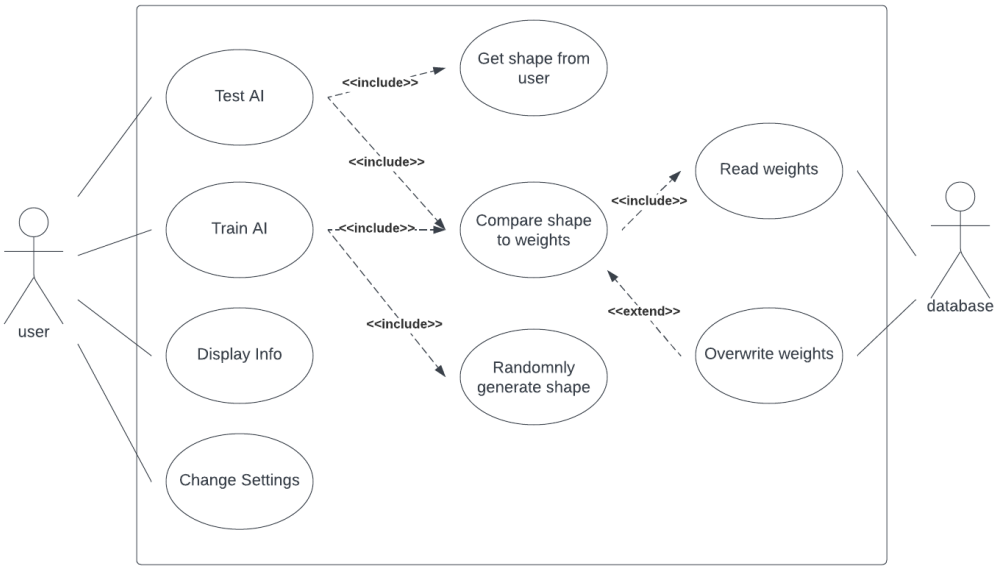
- A database will be used to store the weights of the shapes (a weight value will be stored for each pixel in a 64x64 image)
- A connection to the database is maintained to allow SQL queries to be executed
- SQL queries will be used to retrieve and modify the weights
- Formatting the results of the SQL queries will be used to display the weights of each of the shapes on the screen in the form of an image, with the weight value for each pixel determining the brightness of the pixel.

Constraints:

The constraints that apply to this project are:

- Python and the Pygame module will be used to develop the program.
- The program will only run on a computer that has Python installed along with the Pygame, numpy, math, datetime, random and sqlite3 modules.
- The database file must be kept in an “assets” folder, which must be located in the same directory as the program file.
- Training the AI to a high accuracy will take a very long time and will use a lot of processing power.
- The project must be completed before March 15th.

UML use case diagram:



Requirements Specification:

End-user requirements:

End users will expect:

- To be able to choose to train or test the AI
- To be able to draw a shape using the mouse when testing the AI and for the results of testing the AI to be displayed on the screen
- The total accuracy and the individual accuracies for each kind of shape to be displayed on the screen when training the AI
- An intuitive user interface which is easy to use
- To be able to view info about the program on the info screen
- To be able to change program settings on the settings screen

Functional requirements:

All inputs are either imported from a database file or come from the user and all outputs displayed on the screen.

Inputs:

- Weights (from database)
- A shape drawn by the user
- Mouse clicks on menu buttons
- Custom values for settings set by the user

Processes:

- Read the weights from an external database file
- Get the shape drawn by the end-user
- Randomly generate a shape
- Compare the shape to the weights of all the shapes
- Adjust the shape's weights
- Calculate the accuracies for each kind of shape
- Calculate the total accuracy of the program
- Sort the accuracies of the shapes in order of highest to lowest accuracy
- Sort the results of testing the AI in order of highest to lowest certainty
- Set custom values for program settings

Outputs:

- When training the AI, the total accuracy of the AI and the individual accuracies of each kind of shape will be displayed on screen
- The program will also display the weights in the form of an image, with the brightness of each pixel being determined by the weight value for that pixel (the higher the value, the brighter the pixel) when training the AI
- When testing the AI, the program's guess of what shape the user created will be displayed
- The instructions on how to use the program will be displayed on the info screen

Project Plan:

Main Stage	Sub-Tasks	Estimated time (hrs)	Total time (hrs)
Design	• GUI design • Database design • UML class diagrams • Top level modular design • Design core modules	2 1 1 3 8	15
Implement	• Research knowledge beyond the course • Create/acquire media assets • Code class modules • Test class modules • Code sqlite3 database integration • Test database integration • Code core modules • Test core modules • Code integrated program • Gather evidence of implementation	2 1 2 1 1 1 8 2 15 2	35
Testing	• Test integrated program • Test GUI • Persona and test case testing	2 2 3	7
Evaluation	• Fitness for purpose evaluation • Robustness evaluation • Maintainability evaluation	1 1 1	3

Resources:

Physical resources:

- A computer using Windows 7 or later, with a mouse, a screen with at least 1094x671 pixels and at least 1GB RAM

Software resources:

- Python 3.7 or later, along with the numpy, sqlite3, math, random, datetime and pygame modules, for the implementation
- IDLE for coding the implementation of the project
- Microsoft Word for creating the project documentation
- Microsoft OneDrive for storing the project files to be accessed on different devices
- DB Browser for SQLite for viewing the database and testing that values are saved properly
- Google Chrome for research and for accessing online resources

Online resources:

- Photopea for creating the media assets
- DaFont for acquiring the fonts
- Google Colab for the vast majority of the AI training.
- Google Drive will be used to save the database when it is training
- Trello to organise the tasks in progress/to be done
- Lucidchart to design the UML use case and class diagrams, and the top level design
- Stack Overflow for finding solutions to errors I might have during implementation

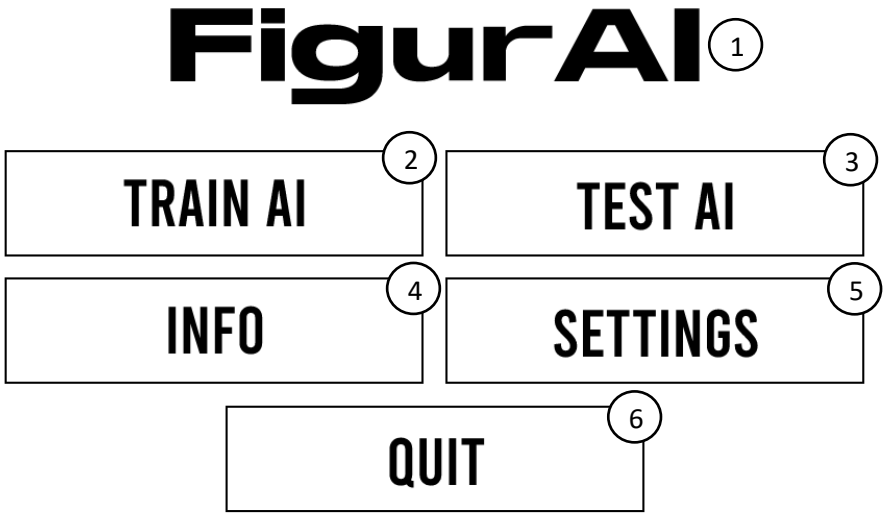
Human resources:

- My teacher for ongoing advice and consultation
- My classmates to help test the program and give advice on screen layouts
- My family and friends to help test the program

Design

GUI Design:

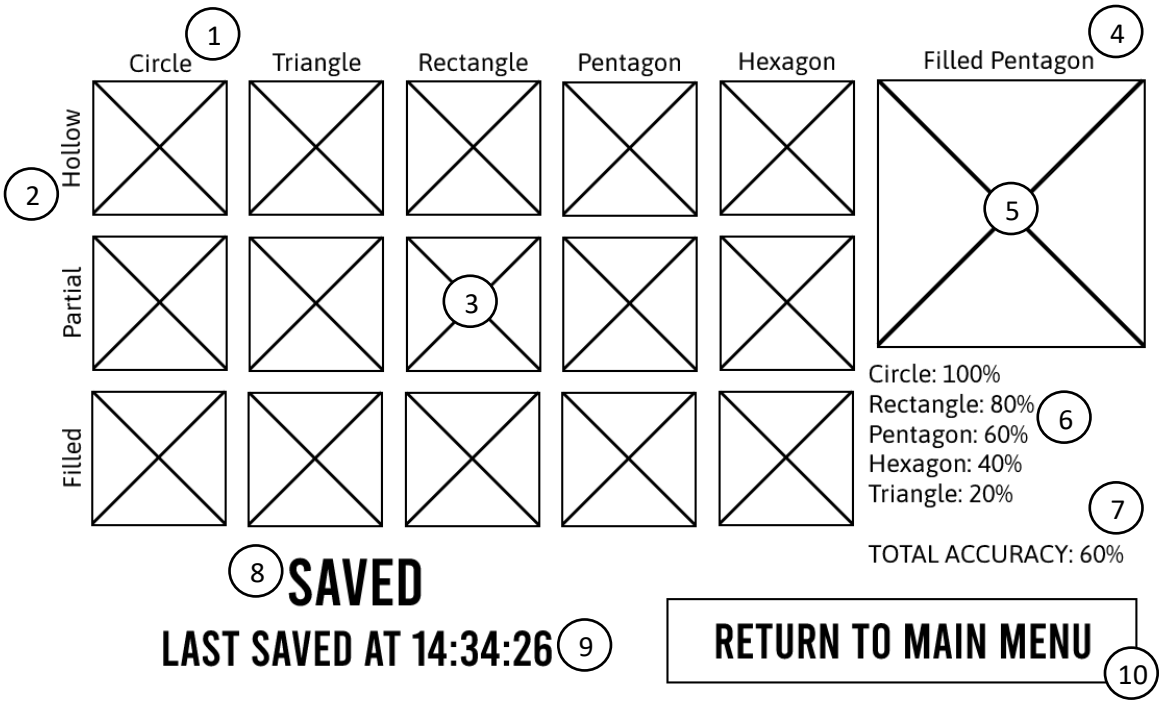
Menu Screen (1094px x 671px)



This screen appears when the program is launched or when the user returns to the main menu from any of the other screens. Background color – Dark grey

1. The name of the program displayed in Stretch Pro font (Size - 100). The “Figur” text is white and the “AI” text is yellow.
2. The “Train AI” button takes the user to the Training Screen and starts training the AI. The text is white and the font is Bebas Neue (Size - 64). The button has a dark grey background and a white outline. The text and the outline will turn yellow when the user hovers the mouse over the button.
3. The “Test AI” button takes the user to the Testing Screen where they can draw their own shape for the program to guess. The text is white and the font is Bebas Neue (Size - 64). The button has a dark grey background and a white outline. The text and the outline will turn yellow when the user hovers the mouse over the button.
4. The “Info” button takes the user to the Info Screen. The text is white and the font is Bebas Neue (Size - 64). The button has a dark grey background and a white outline. The text and the outline will turn yellow when the user hovers the mouse over the button.
5. The “Settings” button takes the user to the Settings Screen. The text is white and the font is Bebas Neue (Size - 64). The button has a dark grey background and a white outline. The text and the outline will turn yellow when the user hovers the mouse over the button.
6. The “Quit” button exits the program. The text is white and the font is Bebas Neue (Size - 64). The button has a dark grey background and a white outline. The text and the outline will turn yellow when the user hovers the mouse over the button.

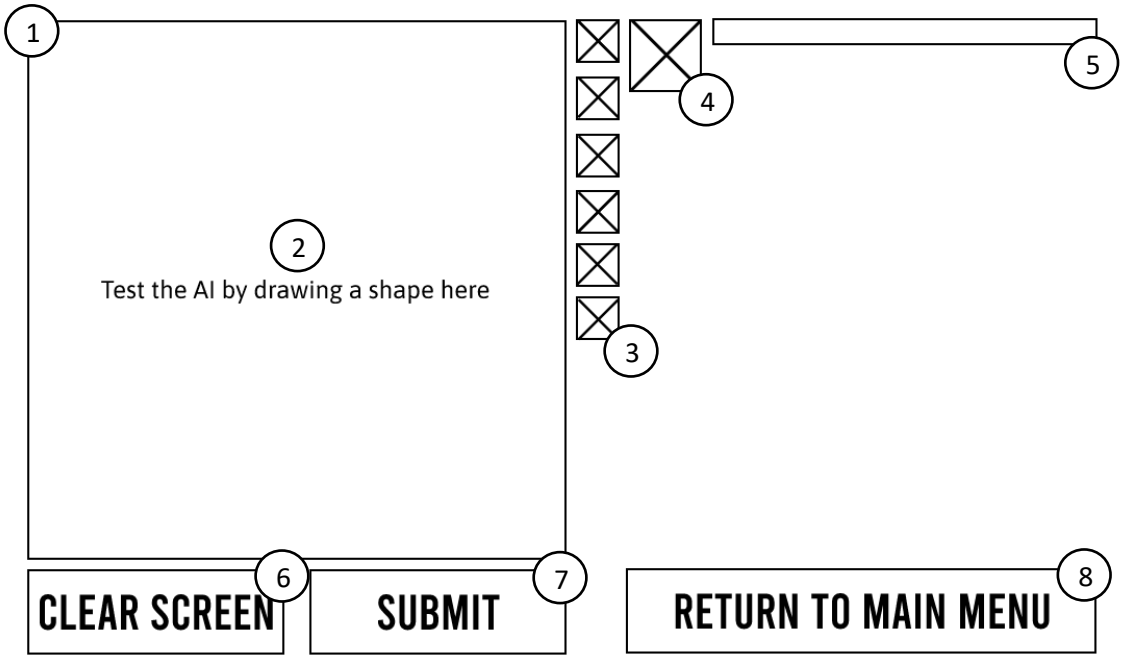
Training Screen (1094px x 671px)



This screen appears when the user clicks on the “Train AI” button on the Menu Screen. Background color – Dark grey

1. The names of the different kinds of shape are displayed in white in Asap font (Size - 24).
2. The names of the different types of shape are displayed in white in Asap font (Size - 24).
3. These are 128px x 128px image representations of the weights of the shapes/shape types. For each pixel in the image, the higher the weight value the brighter the pixel.
4. The shape type and shape name of a randomly selected shape that the program has recently generated is displayed in white in Asap font (Size - 24).
5. The mini shape display (256px x 256px) has a dark grey background and a white outline. A random shape that has been recently generated will be displayed in white.
6. These are the accuracies of the shapes displayed in descending order of accuracy in white in Asap font (Size - 24).
7. The total accuracy of the program calculated from the average of all the individual shape accuracies displayed in white in Asap font (Size - 24).
8. The save status of the program displayed in white in Bebas Neue font (Size - 64).
9. The time of the last save displayed in white in Bebas Neue font (Size - 48).
10. The “Return to Main Menu” button returns the user to the main menu. The text is white and the font is Bebas Neue (Size - 48). The button has a dark grey background and a white outline. The text and the outline will turn yellow when the user hovers the mouse over the button.

Testing Screen (1094px x 671px)

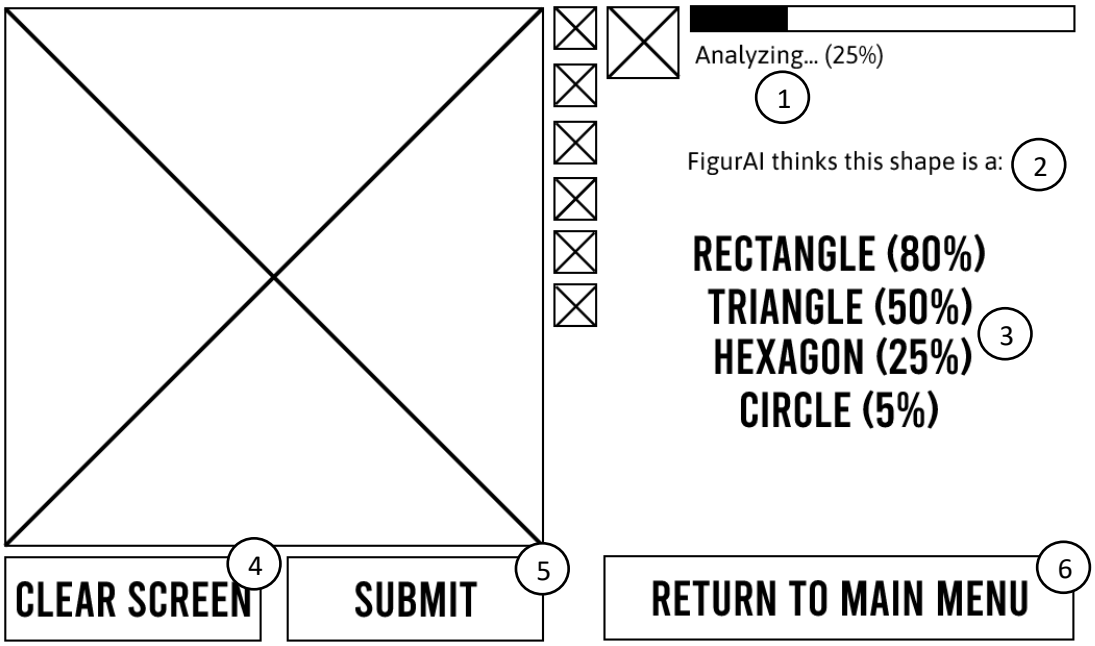


This screen appears when the user clicks on the “Test AI” button on the Menu Screen. Background color – Dark grey

1. This box (512px x 512px) is where the user draw a shape (freehand) using the mouse. The background of the box will be dark grey and the outline will be white.
2. The instruction for the user is displayed in white in Asap font (Size - 24). When the user starts drawing in the drawing box, this text will disappear and will re-appear when the user clicks on the “Clear Screen” button.
3. The drawing controls allow the user to select what happens when they draw in the drawing box. There is an undo button, 3 brush sizes, an eraser and a fill tool. The background color of the buttons is dark grey. The color of the outline and the image in the middle of the button is white and turns yellow when the user hovers the mouse over the button and remains yellow when the button is clicked, until a different drawing option is selected and the color will return to white.
4. The mini shape display (64px x 64px) has a dark grey background and a white outline. When the user clicks the “Submit” button, the display shows a downscaled version of the shape the user has drawn as the program rotates it and analyzes it. When the user clicks the “Clear Screen” button this display will return to dark grey.
5. The progress bar shows the progress of the program rotating and analyzing the shape the user has drawn. The bar has a white outline and as the analyzing progress increases, the bar will fill up with green. The bar is dark grey until the user clicks on the “Submit” button. When the user clicks the “Clear Screen” button this bar will return to dark grey.
6. The “Clear Screen” button resets the drawing box back to dark grey and deletes the shape the user has drawn. This button is inactive whenever the drawing box is empty. When the user starts drawing in the drawing box, the button becomes active again and can be clicked. The font is Bebas Neue (Size - 48) and the background color is dark grey. When the button is inactive, the text and outline color is light grey. When the button is active, the text and the outline color is white and will turn yellow when the user hovers the mouse over the button.

7. The “Submit” button. This button is inactive whenever the drawing box is empty. When the user starts drawing in the drawing box, the button becomes active again and can be clicked. The font is Bebas Neue (Size - 48) and the background color is dark grey. When the button is inactive, the text and outline color is light grey. When the button is active, the text and the outline color is white and will turn yellow when the user hovers the mouse over the button.
8. The “Return to Main Menu” button returns the user to the main menu. The text is white and the font is Bebas Neue (Size - 48). The button has a dark grey background and a white outline. The text and the outline will turn yellow when the user hovers the mouse over the button.

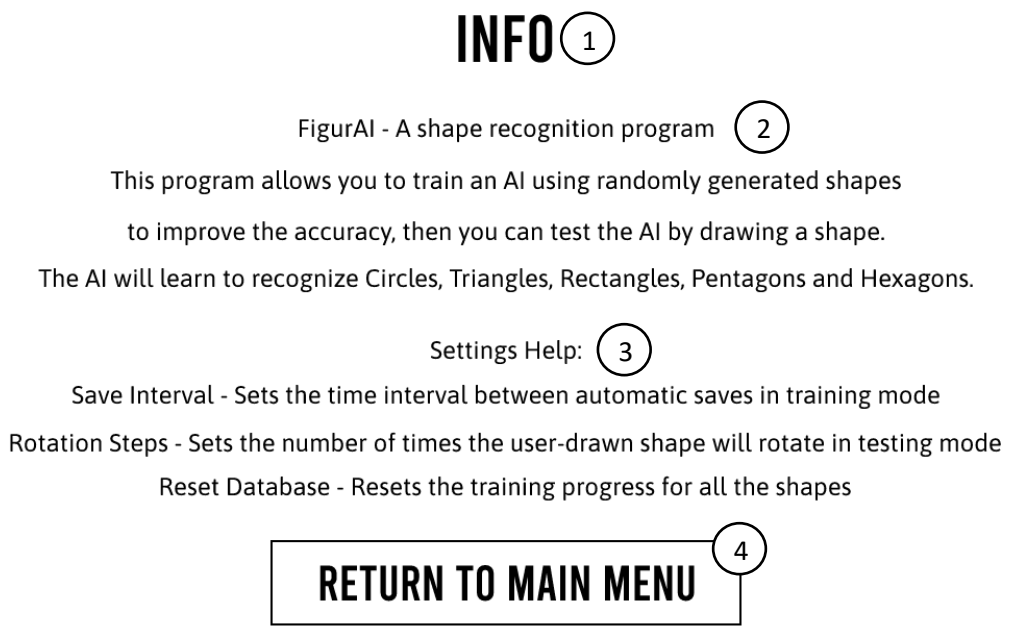
Testing Screen (Analyzing Shape) (1094px x 671px)



This screen is an extension of the Testing Screen, with elements that appear when the user clicks on the “Submit” button on the Testing Screen. Background color – Dark grey

1. This text appears when the use clicks on the “Submit” button. The percentage shows the progress of the program analyzing the user-drawn shape. The text color is white and the font is Asap (Size - 24).
2. This text appears when the use clicks on the “Submit” button. The text color is white and the font is Asap (Size - 24).
3. The names of the shape which the program thinks the user has drawn which appear when the use clicks on the “Submit” button. The percentage shows how certain the program is that the shape is correct. As the program analyzes the shape, the shapes it thinks the user has drawn will change and will be sorted in order of highest percentage certainty to lowest. The text color is white and the font is Bebas Neue (Size - 48). When the program finishes analyzing the shape, the shape name with the highest percentage certainty will turn yellow.
4. When the program is analyzing the user-drawn shape, the “Clear Screen” button will cancel the analysis and clear the screen.
5. When the program is analyzing the user-drawn shape, the “Submit” button is inactive and cannot be clicked. After the program finishes analyzing the shape, the button will remain inactive until the user makes any changes to the shape in the drawing box.
6. When the program is analyzing the user-drawn shape, the “Return to Main Menu” button will cancel the analysis and return the user to the main menu.

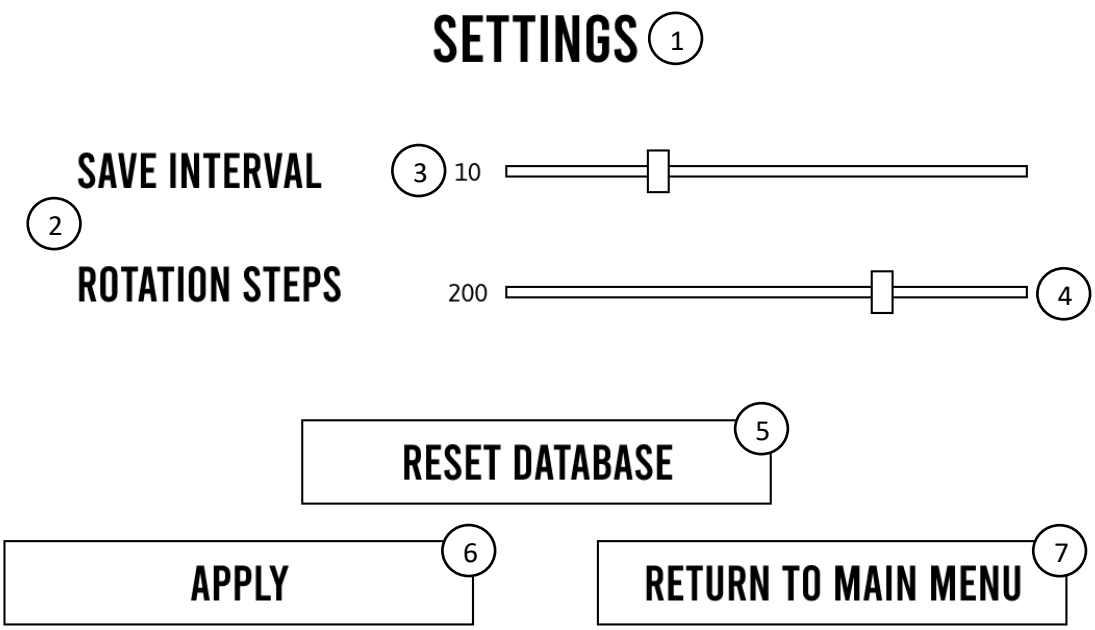
Info Screen (1094px x 671px)



This screen appears when the user clicks on the “Info” button on the Menu Screen. Background color – Dark grey

1. The Info heading displayed in white in Bebas Neue font (Size - 64)
2. A brief description on what the program is and how to use it. The text color is white and the font is Asap (Size - 24)
3. A brief description of the different settings that can be changed on the Settings screen. The text color is white and the font is Asap (Size - 24)
4. The “Return to Main Menu” button returns the user to the main menu. The text is white and the font is Bebas Neue (Size - 48). The button has a dark grey background and a white outline. The text and the outline will turn yellow when the user hovers the mouse over the button.

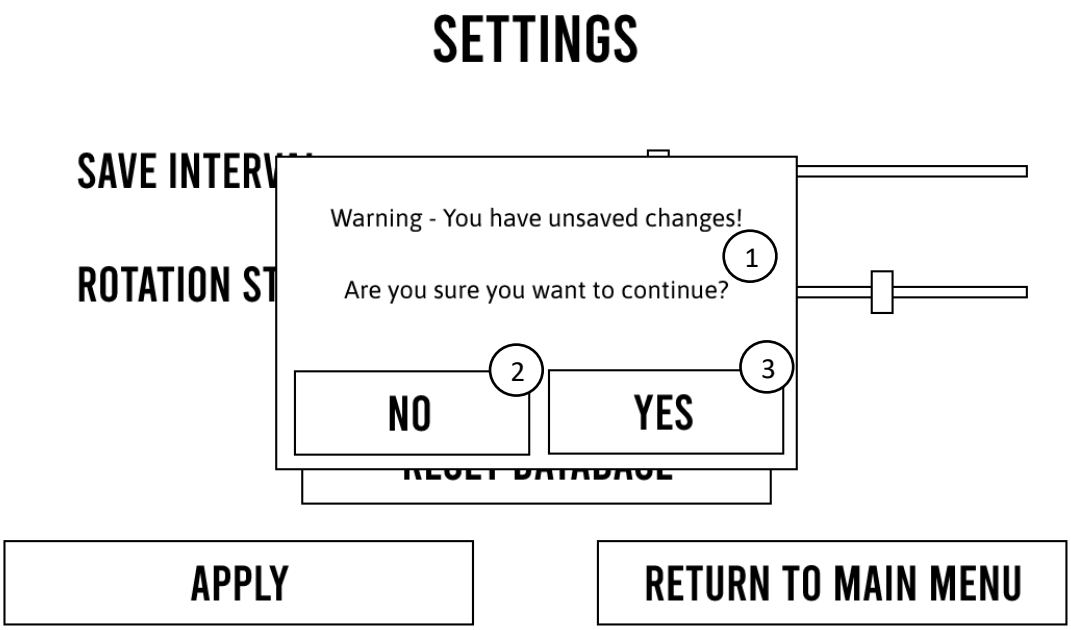
Settings Screen (1094px x 671px)



This screen appears when the user clicks on the “Settings” button on the Menu Screen. Background color – Dark grey

1. The Settings heading displayed in white in Bebas Neue font (Size - 64)
2. The names of the program settings displayed in Bebas Neue font (Size - 48)
3. The current value of the slider which changes as the user changes the position of the slider handle. The text color is white and the font is Asap (Size - 24)
4. The sliders which control the values of the settings. As the user drags the handle to the right, the value for that setting increases, and as the user drags the handle to the left it decreases. The color of the slider bar is light grey, and the color of the handle is dark grey with a white outline.
5. The “Reset Database” button opens a warning popup and prompts the user if they wish to continue. If they do, then the database will be reset and all the weights will be set to 0. The text is white and the font is Bebas Neue (Size - 48). The button has a dark grey background and a white outline. The text and the outline will turn yellow when the user hovers the mouse over the button.
6. The “Apply” button saves the changes the user has made to the settings. The button is inactive and cannot be used until the user has changed the position of any of the sliders. The font is Bebas Neue (Size - 48) and the button has a dark grey background. When the button is inactive, the text and the outline color is light grey. When the button is active, the text and the outline color is white and will turn yellow when the user hovers the mouse over the button. When the user clicks the button, it will become inactive until the sliders are changed.
7. The “Return to Main Menu” button returns the user to the main menu unless they have made unsaved changes, in which case a warning popup appears and prompts the user if they wish to continue. If they do, they will be returned to the main menu and the changes will be lost. The text is white and the font is Bebas Neue (Size - 48). The button has a dark grey background and a white outline. The text and the outline will turn yellow when the user hovers the mouse over the button.

Settings Screen (Warning Popup) (1094px x 671px)



This popup appears when the user clicks on the “Reset Database” button on the Settings Screen or if the user clicks on the “Return to Main Menu” button on the Settings Screen and the user has made unsaved changes to the settings. Background color – Dark grey

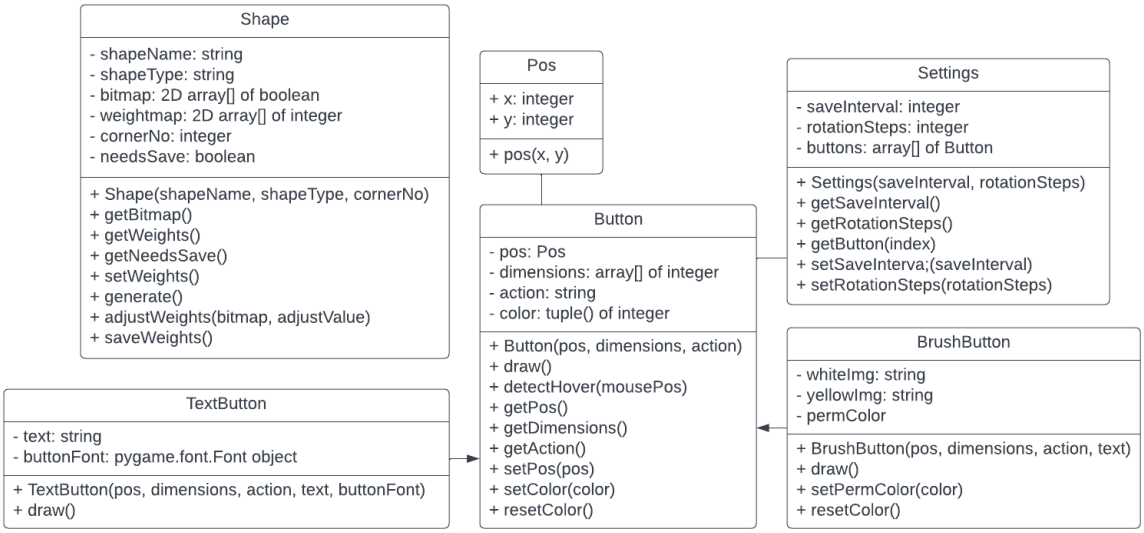
1. The warning message displayed in white in Asap font (Size - 24)
2. The “No” button hides the warning popup and cancels the action the user was trying to do. The text is white and the font is Bebas Neue (Size - 48). The button has a dark grey background and a white outline. The text and the outline will turn yellow when the user hovers the mouse over the button.
3. The “Yes” button continues the action the user was trying to do. The text is white and the font is Bebas Neue (Size - 48). The button has a dark grey background and a white outline. The text and the outline will turn yellow when the user hovers the mouse over the button.

Justifications:

I have chosen a primarily dark color scheme, with dark grey as the background color for all the screens, because I find that dark color schemes are easier to look at than bright color schemes, and lack of colorful elements matches the simplicity of the program. Also, this program is intended to run for long periods of time when training the AI, so using dark colors saves energy and creates less background light to the area that the computer is in. I chose white and yellow as the main colors for the elements on screen because they are similar in brightness, and they both stand out from the dark grey background. For the fonts, I chose Stretch Pro for the title because it is eye-catching, bold and very wide, so it balances out with the wide buttons and takes up more of the screen. I chose Bebas Neue as the font for the buttons because it is tall, which makes it space efficient and allows more text to fit onto the buttons. This text is used elsewhere to keep a consistent theme with the buttons. I chose Asap because it is easy to read when the font size is small, which allows lots of information to be easily read. All fonts are sans serif to match the simplicity of the program.

Project Design:

UML Class diagram:



Data Structures:

Data Structure: Array of objects

Purpose: To store a TextButton or BrushButton object for each of the buttons the user can click on for each screen

Implementation: Python array of a mix of TextButton and BrushButton objects as defined in the UML class diagram

Example (Testing Screen):

Index	Content
0	TextButton(this.pos = pos, this.dimensions = (width, height), this.action = "exit_menu", this.text = "Return to Main Menu", this.buttonFont = font)
1	TextButton(this.pos = pos, this.dimensions = (width, height), this.action = "clear", this.text = "Clear Screen", this.buttonFont = font)
2	TextButton(this.pos = pos, this.dimensions = (width, height), this.action = "submit", this.text = "Submit", this.buttonFont = font)
3	BrushButton(this.pos = pos, this.dimensions = (width, height), this.action = "undo")
4	BrushButton(this.pos = pos, this.dimensions = (width, height), this.action = "brush_8")
5	BrushButton(this.pos = pos, this.dimensions = (width, height), this.action = "brush_12")
6	BrushButton(this.pos = pos, this.dimensions = (width, height), this.action = "brush_16")
7	BrushButton(this.pos = pos, this.dimensions = (width, height), this.action = "brush_eraser")
8	BrushButton(this.pos = pos, this.dimensions = (width, height), this.action = "brush_fill")

Data Structure: Associative array of objects

Purpose: To store a Shape object for each of the kinds of shape the program can generate

Implementation: Python dictionary of Shape objects as defined in the UML class diagram

Index	Content
Hollow_Circle	Shape(this.shapeName = “Circle”, this.shapeType = “Hollow”, this.cornerNo = 0)
Partial_Circle	Shape(this.shapeName = “Circle”, this.shapeType = “Partial”, this.cornerNo = 0)
Filled_Circle	Shape(this.shapeName = “Circle”, this.shapeType = “Filled”, this.cornerNo = 0)
Hollow_Triangle	Shape(this.shapeName = “Triangle”, this.shapeType = “Hollow”, this.cornerNo = 3)
Partial_Triangle	Shape(this.shapeName = “Triangle”, this.shapeType = “Partial”, this.cornerNo = 3)
Filled_Triangle	Shape(this.shapeName = “Triangle”, this.shapeType = “Filled”, this.cornerNo = 3)
Hollow_Rectangle	Shape(this.shapeName = “Rectangle”, this.shapeType = “Hollow”, this.cornerNo = 4)
Partial_Rectangle	Shape(this.shapeName = “Rectangle”, this.shapeType = “Partial”, this.cornerNo = 4)
Filled_Rectangle	Shape(this.shapeName = “Rectangle”, this.shapeType = “Filled”, this.cornerNo = 4)
Hollow_Pentagon	Shape(this.shapeName = “Pentagon”, this.shapeType = “Hollow”, this.cornerNo = 5)
Partial_Pentagon	Shape(this.shapeName = “Pentagon”, this.shapeType = “Partial”, this.cornerNo = 5)
Filled_Pentagon	Shape(this.shapeName = “Pentagon”, this.shapeType = “Filled”, this.cornerNo = 5)
Hollow_Hexagon	Shape(this.shapeName = “Hexagon”, this.shapeType = “Hollow”, this.cornerNo = 6)
Partial_Hexagon	Shape(this.shapeName = “Hexagon”, this.shapeType = “Partial”, this.cornerNo = 6)
Filled_Hexagon	Shape(this.shapeName = “Hexagon”, this.shapeType = “Filled”, this.cornerNo = 6)

Data Structure: Array of objects

Purpose: To store a 2D array object (to store the pixels in the image) for the state of the shape the user has drawn every time the user changes it (allows the user to use the undo button to go back to a previous version of the shape).

Implementation: Python array of numpy.ndarray objects as defined in the numpy package

Example (user has drawn in the drawing box 3 times):

Index	Content
0	2D numpy array
1	2D numpy array
2	2D numpy array

Database Design:

There are 15 tables, one for each weight (Hollow Circle, Partial Circle, Filled Circle, Hollow Triangle, Partial Triangle, Filled Triangle, Hollow Rectangle, Partial Rectangle, Filled Rectangle, Hollow Pentagon, Partial Pentagon, Filled Pentagon, Hollow Hexagon, Partial Hexagon, Filled Hexagon). These tables all have the same format so the data dictionary, SELECT query design and INSERT query design is the same for all tables.

Data Dictionary (Repeated for all tables):

Attribute name	Key	Type	Required	Validation
xIndex		integer	yes	
yIndex		integer	yes	
weightValue		integer	yes	

SELECT query design (Repeated for all tables):

Fields	xIndex, yIndex, weightValue
Search Criteria	None
Grouping	None
Sort Order	None

These are used to retrieve the weights from the database

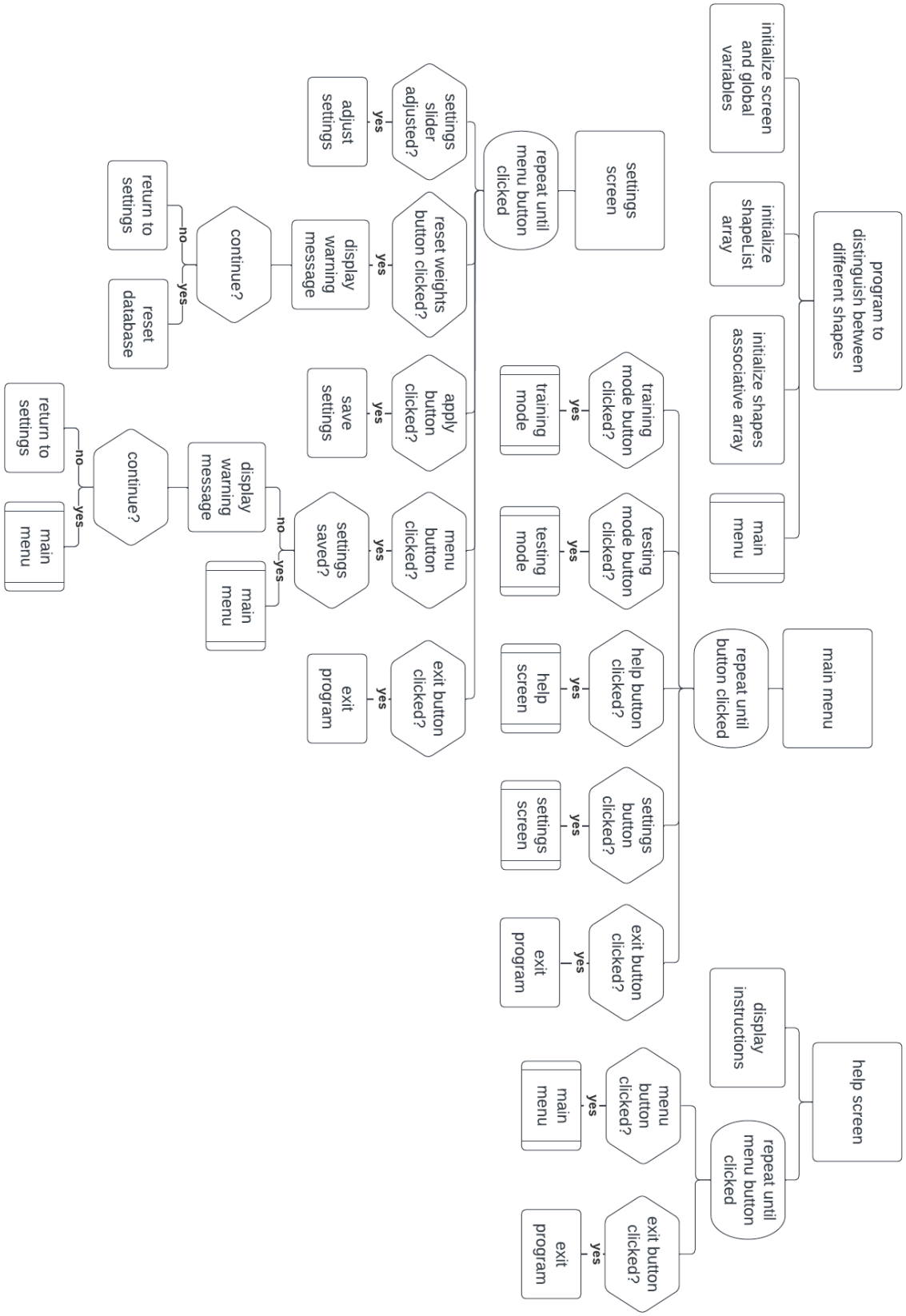
INSERT query design (Repeated for all tables):

Fields	xIndex, yIndex, weightValue
Data Values	x, y, weightmap[x, y]

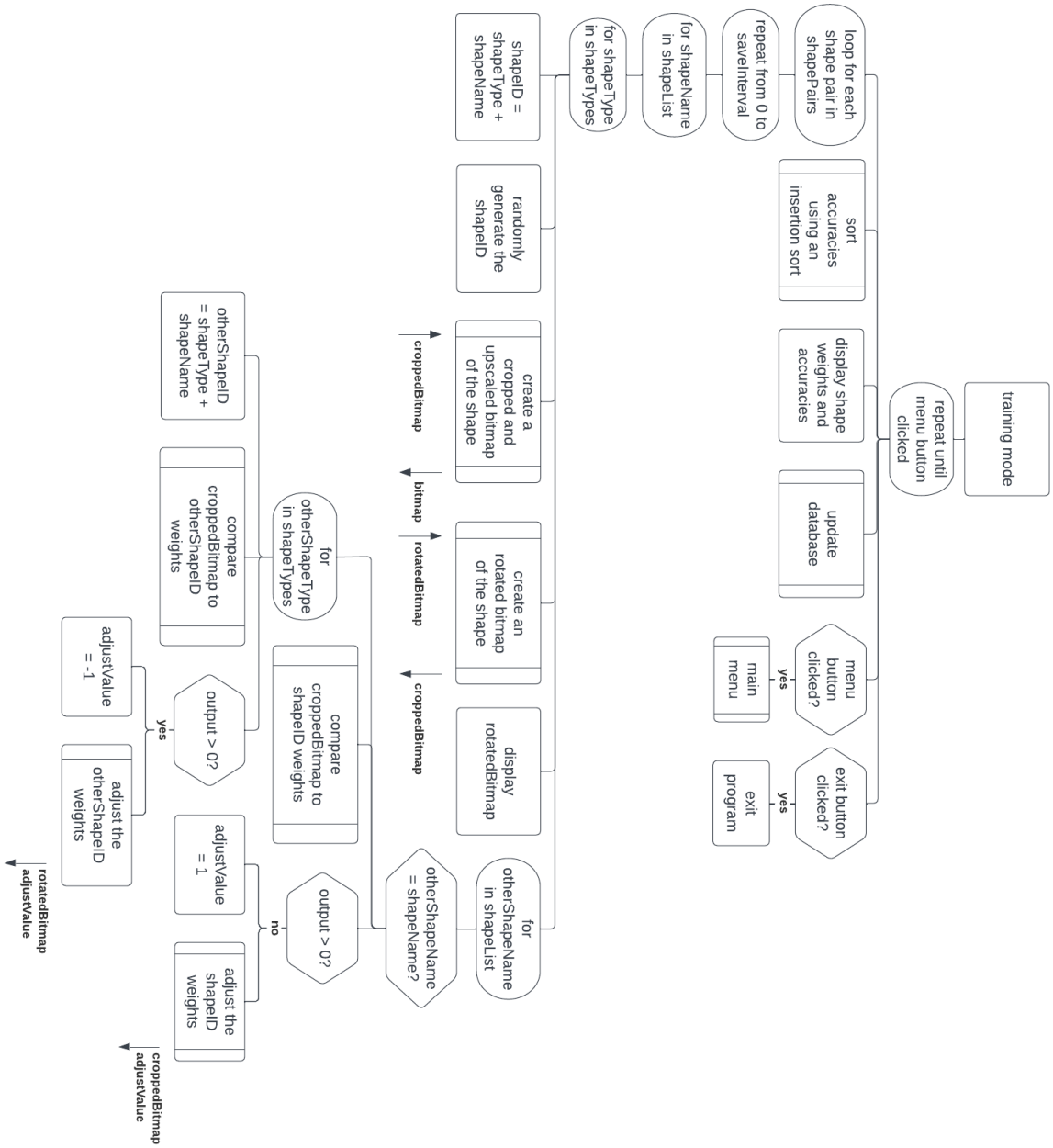
These are used to save the updated weights into the database

Structure Diagrams:

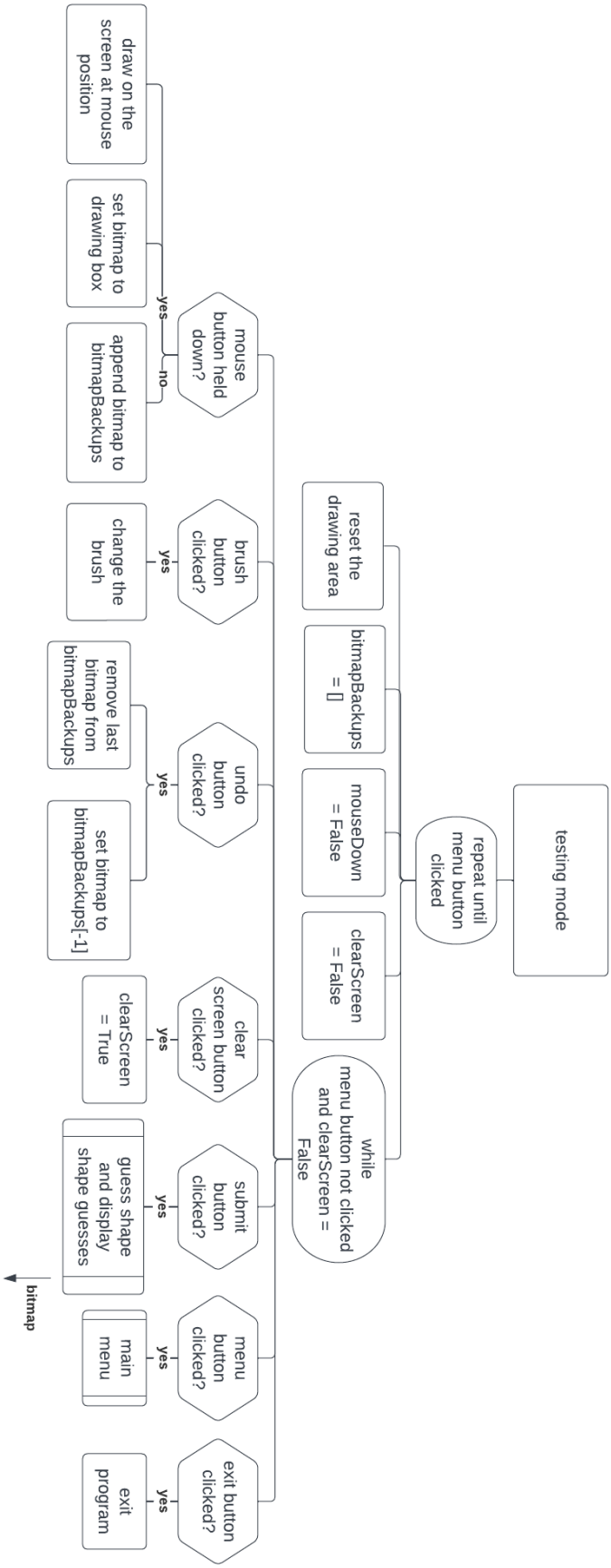
Main program, main menu, help screen and settings screen structure diagrams:



Training mode structure diagram:



Testing mode structure diagram:



Refinements:

Insertion sort algorithm:

```
DEFINE FUNCTION insertionSort(ARRAY array):
   FOR i FROM 1 TO len(array) - 1 DO:
      SET currentValue TO array[i]
      SET pos TO i
      WHILE pos > 0 AND array[pos - 1][1] < currentValue[1] DO:
         SET array[pos] TO array[pos - 1]
         SET pos TO pos – 1
      END WHILE
      SET array[pos] TO currentValue
   END FOR
   RETURN array
END FUNCTION
```

Initialize Shape object:

```
DEFINE CONSTRUCTOR Shape(STRING shapeName, STRING shapeType, INT cornerNo):
   SET THIS.__shapeName TO shapeName
   SET THIS.__shapeType TO shapeType
   SET THIS.__cornerNo TO cornerNo
   SET THIS.__bitmap TO <empty 256x256 2D array of BOOL>
   SET THIS.__weightmap TO <empty 64x64 2D array of INT>
   SET THIS.__needsSave TO FALSE
END CONSTRUCTOR
```

Initialize Button object:

```
DEFINE CONSTRUCTOR Button(OBJECT pos, TUPLE dimensions, STRING action):
   SET this.__pos TO pos
   SET this.__dimensions TO dimensions
   SET this.__action TO action
   SET this.__color TO (255, 255, 255)
END CONSTRUCTOR
```

Initialize TextButton object:

```
DEFINE CONSTRUCTOR TextButton(OBJECT pos, TUPLE dimensions, STRING action, STRING text,
OBJECT buttonFont):
   SUPER().__init__(pos, dimensions, action)
   SET THIS.__text TO text
   SET THIS.__font TO buttonFont
   THIS.draw()
END CONSTRUCTOR
```

Initialize BrushButton object:

```
DEFINE CONSTRUCTOR BrushButton(OBJECT pos, TUPLE dimensions, STRING action):
   SUPER().__init__(pos, dimensions, action)
   SET THIS.__whiteImg TO <load white image>
   SET THIS.__yellowImg TO <load yellow image>
   SET THIS.__permColor TO (255, 255, 255)
   THIS.draw()
END CONSTRUCTOR
```

Read weights from database:

```
DEFINE FUNCTION readWeights(STRING tableName):
   SET weightmap TO <empty 64x64 2D array of INT>
   <open connection to database>                                    OUT: dbConn
   IF dbConn EXISTS DO:
      SET cur TO dbConn.cursor()
      CREATE TABLE tableName (if it doesn't already exist):
         xIndex INT NOT NULL
         yIndex INT NOT NULL
         weightValue INT NOT NULL
      READ EVERYTHING FROM tableName INTO results
      IF len(results) = weightRes ** 2 DO:
         FOR row IN results:
            SET weightmap[row[0], row[1]] TO row[2]
         END FOR
      ELSE:
         IF len(results) = 0 DO:
            <display unable to find table error message>
         ELSE:
            <display table not correctly formatted error message>
         END IF
         DELETE EVERYTHING IN tableName
         FOR y FROM 0 TO weightRes – 1 DO:
            FOR x FROM 0 TO weightRes – 1 DO:
               INSERT INTO tableName:
                  SET xIndex TO x
                  SET yIndex TO y
                  SET weightValue TO 0
            END FOR
         END FOR
         <display successfully created table message>
      END IF
      <close connection to database>
   END IF
   RETURN weightmap
END FUNCTION
```

Update database:

```
DEFINE FUNCTION updateDB(STRING tableName, 2D ARRAY weightmap):
   <display saving message>
   <open connection to database>                                    OUT: dbConn
   IF dbConn EXISTS DO:
      DELETE EVERYTHING IN tableName
      FOR y FROM 0 TO weightRes – 1 DO:
         FOR x FROM 0 TO weightRes – 1 DO:
            INSERT INTO tableName:
               SET xIndex TO x
               SET yIndex TO y
               SET weightValue TO weightmap[x, y]
         END FOR
      END FOR
      <close connection to database>
      <display successfully saved message>
   END IF
END FUNCTION
```

Open connection to database:

```
DEFINE FUNCTION connectToDB():
   SET conn TO None
   TRY:
      SET conn TO sql.connect(dbFileName)
   EXCEPT sql.Error:
      <display error message>
   END TRY
   IF conn DOES NOT EXIST:
      <display error message>
   END IF
   RETURN conn
END FUNCTION
```

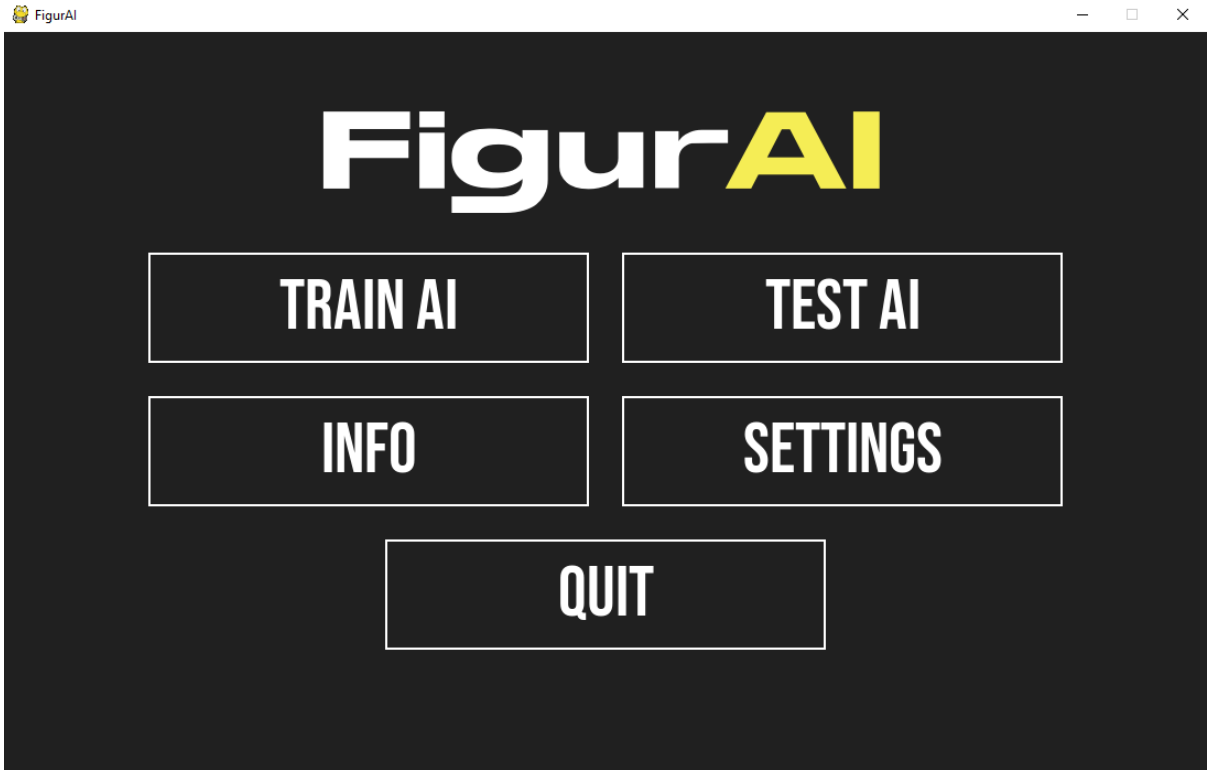
Initialize global shapes array (associative):

```
SET shapes TO {}
FOR i FROM 0 TO len(shapeList) – 1 DO:
   SET shapeName TO shapeList[i]
   FOR shapeType IN shapeTypes DO:
      SET shapeID TO shapeType + shapeName
      SET shapes[shapeID] TO Shape(shapeName, shapeType, i + 2)
      SET weightmap TO readWeights(shapeID)                IN: shapeID, OUT: weightmap
      shapes[shapeID].setWeights(weightmap)
   END FOR
END FOR
```

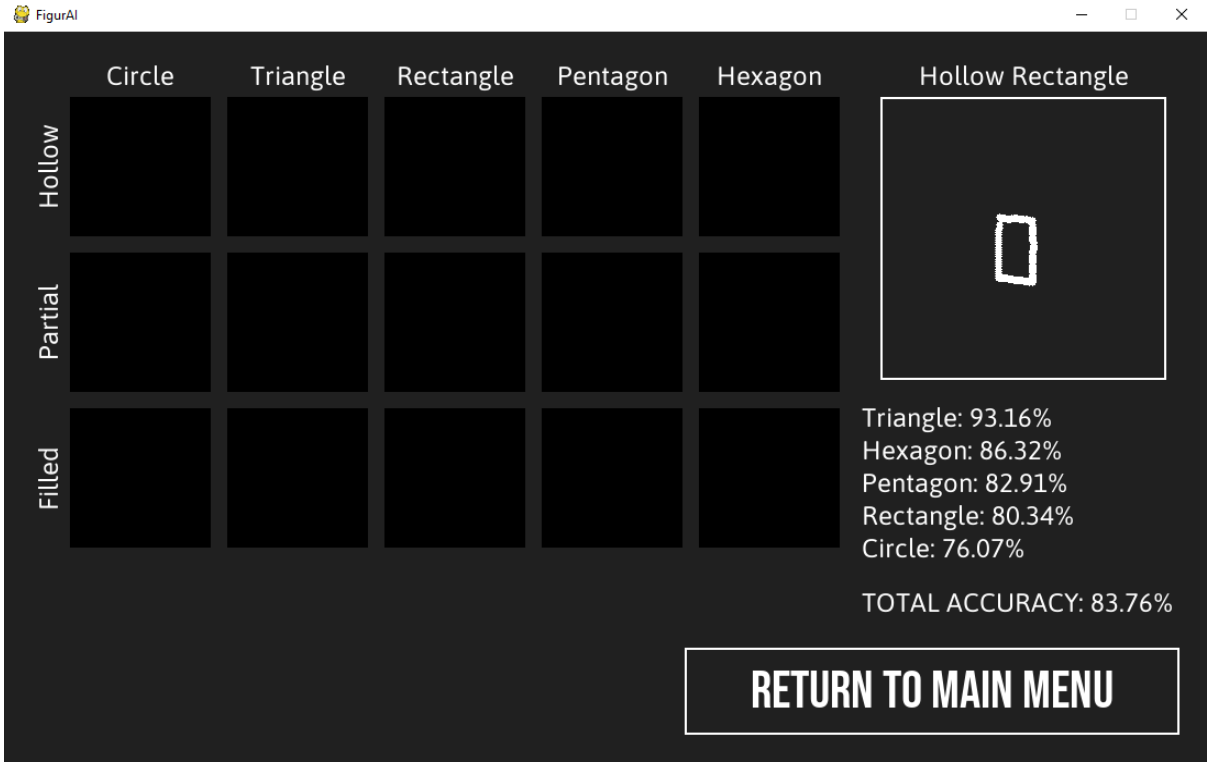
Implementation

Evidence:

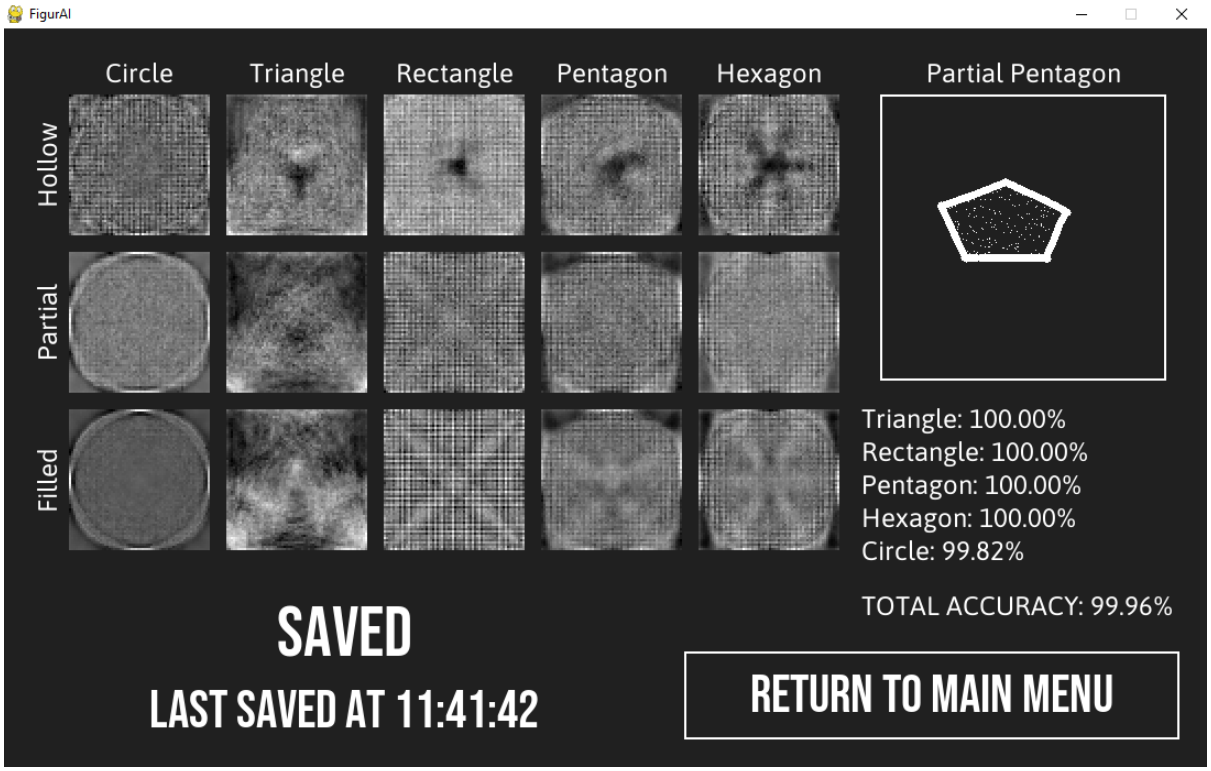
Main Menu:



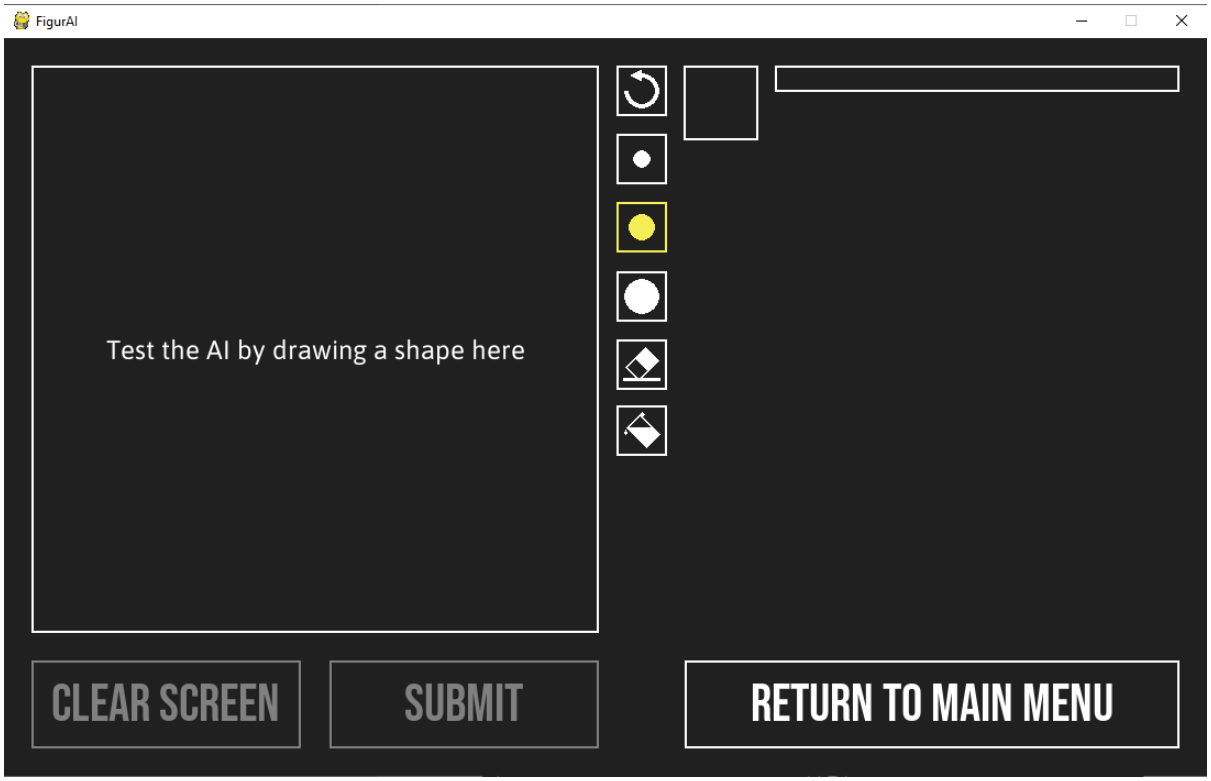
Training (untrained):



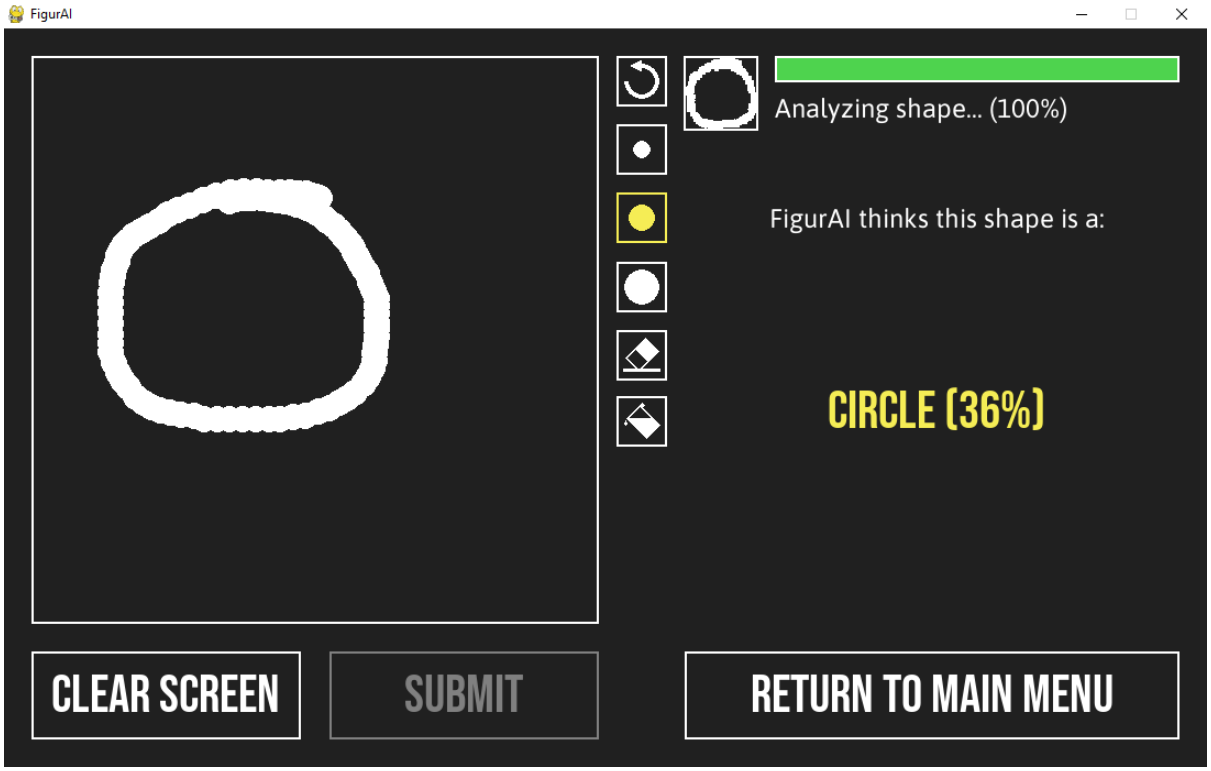
Training Screen (trained a lot):



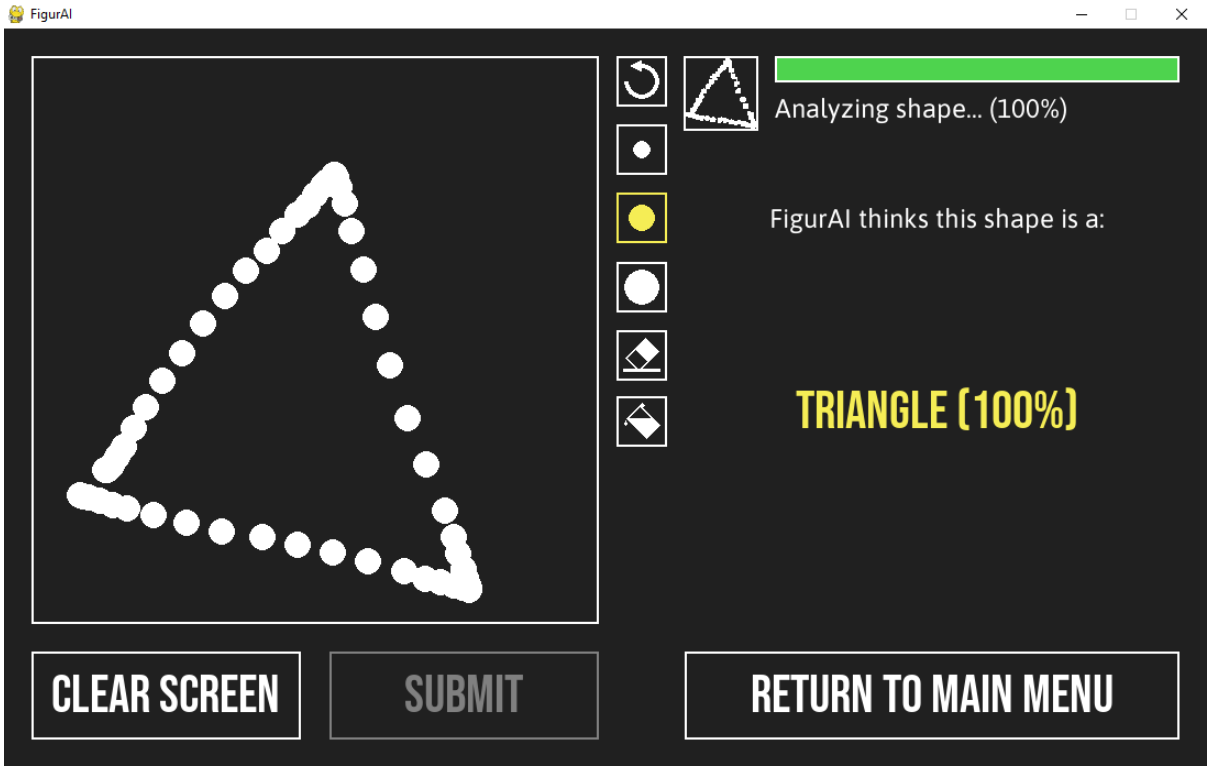
Testing Screen:



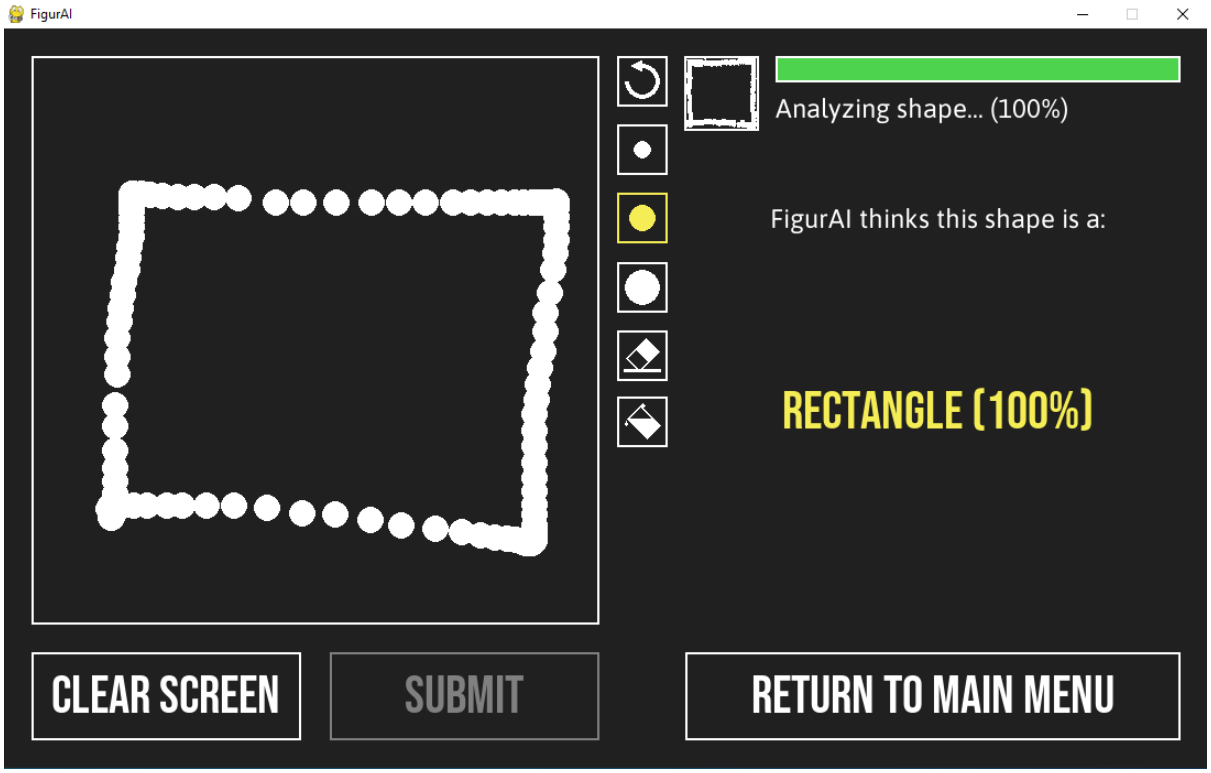
Training Screen (1):



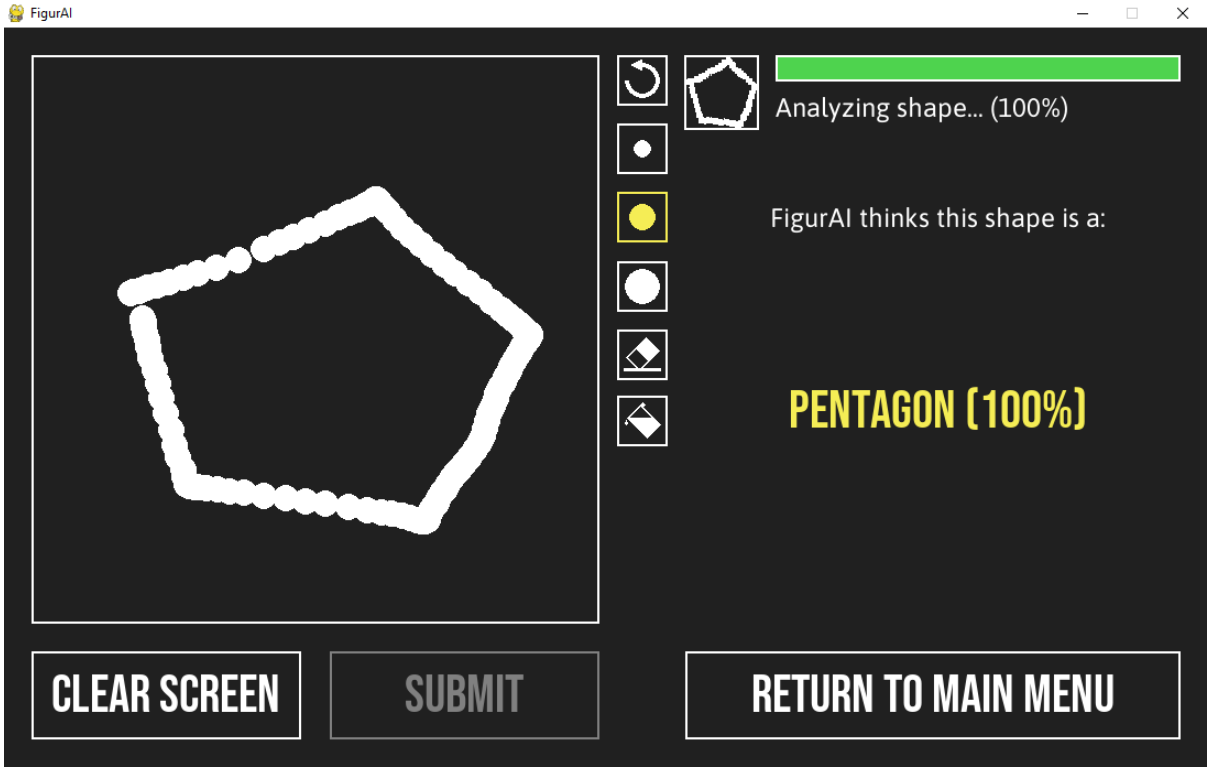
Training Screen (2):



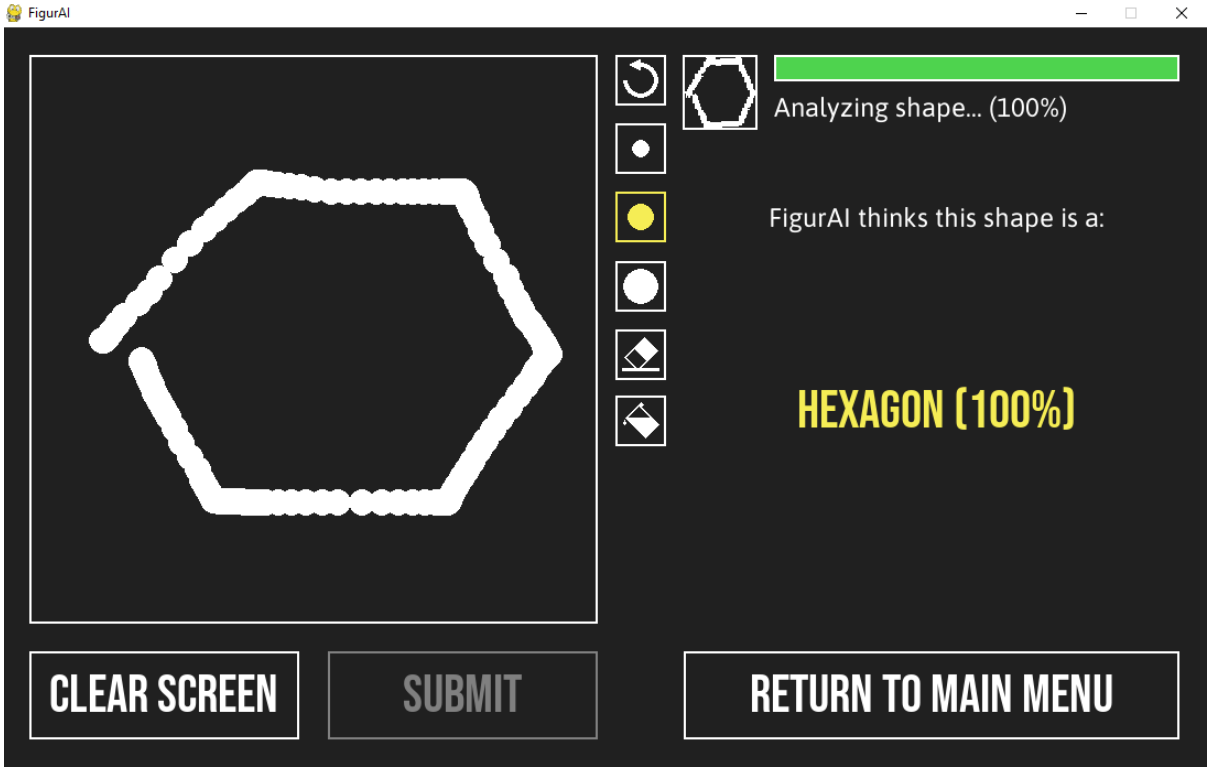
Training Screen (3):



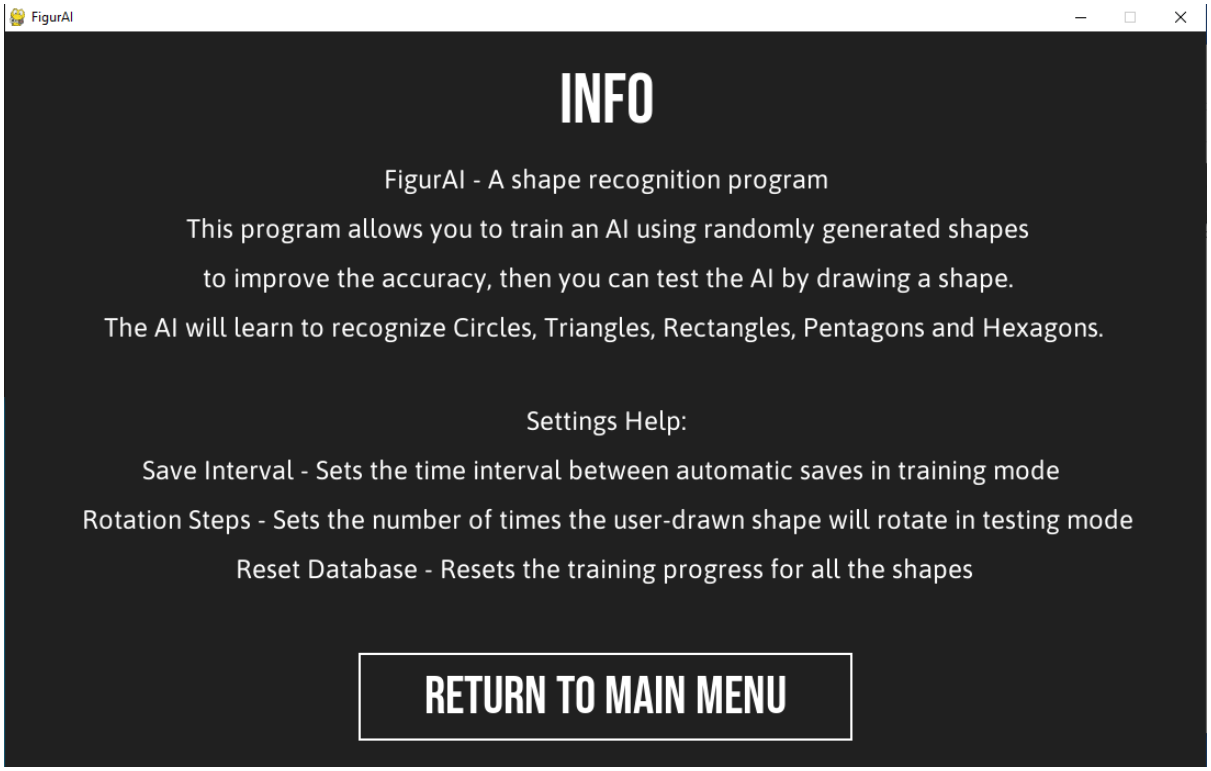
Training Screen (4):



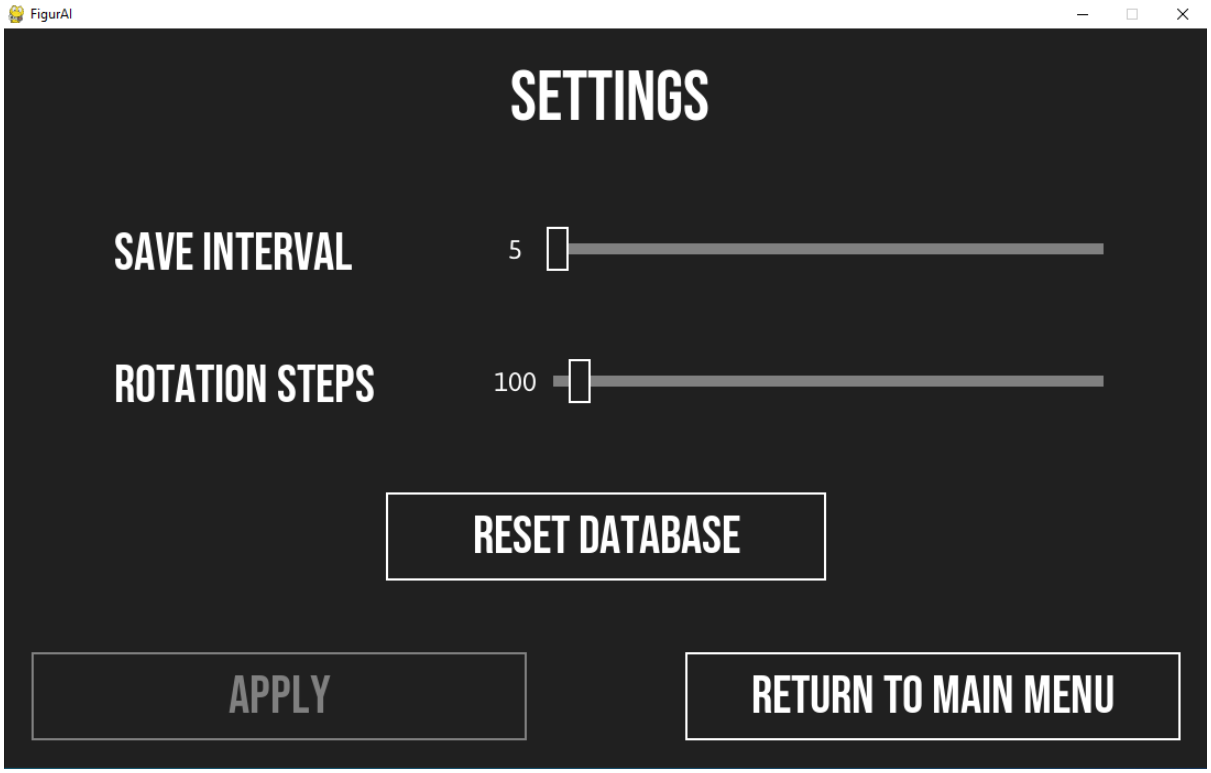
Training Screen (5):



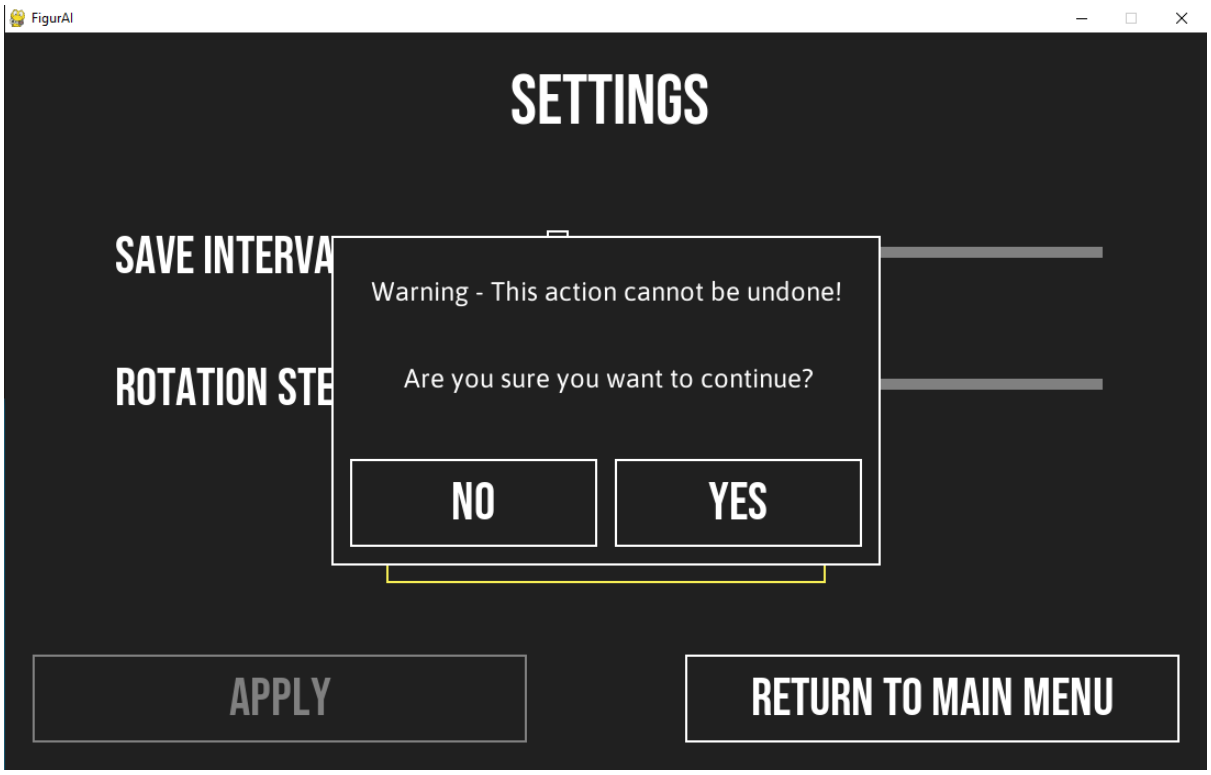
Info Screen:



Settings Screen:



Settings Screen (Warning Popup):



Testing

End-user requirement testing:

Persona:

William Nye is an incredibly handsome SQA marker in his late 30's who has a moderate amount of computing experience. He would like to try out FigurAI as he is interested in playing around with new AI programs to see what they are capable of.

Tasks:

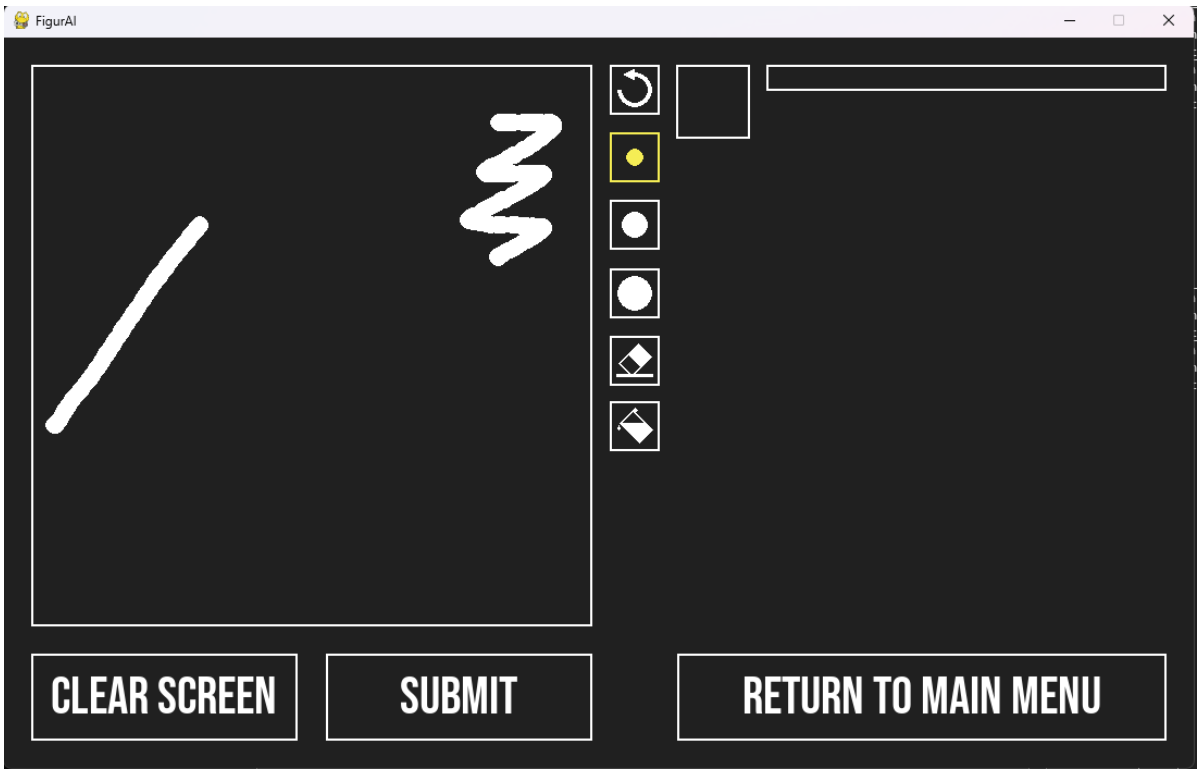
Test no.	End-user requirement being tested	Description of task	Result and comments on process
1	Viewing program Info	Navigate to the Info Screen from the Main Menu	Success
2	Testing the AI	Return to the Main Menu and navigate to the Testing Screen and draw a triangle in the drawing box (using the mouse) with the pen, eraser and fill tools provided. After drawing the shape, click the "Submit" button to see what the program thinks the shape is.	Success – I used the small pen tool and the fill tool to draw a filled in triangle, and I used the eraser tool to give it extra pointy corners. After submitting the drawing, the results were: Triangle (100%), Rectangle (27%). I was a little confused at how the percentages added up to be over 100%, but overall the test was a success.
3	Changing Settings	Return to the Main Menu and navigate to the Settings Screen. Set the save interval to 20, set rotation steps to 1000 and reset the database.	Success
4	Training the AI	Return to the Main Menu and navigate to the Training Screen and wait for the total accuracy to reach 95%.	Success – when I tried returning to the main menu, I forgot to save my settings, but a warning popup appeared allowing me to go back and save them.
5	Testing the AI again	Repeat test 2, try to recreate the shape you drew before.	Success – This time it took much longer for the program to fully analyze the drawing, and results were as follows: Triangle (60%), Rectangle (33%), Pentagon (21%), Hexagon (12%).
6	Exiting the program	Return to the Main Menu and exit the program using the "Quit" button.	Success

Functional requirement testing:

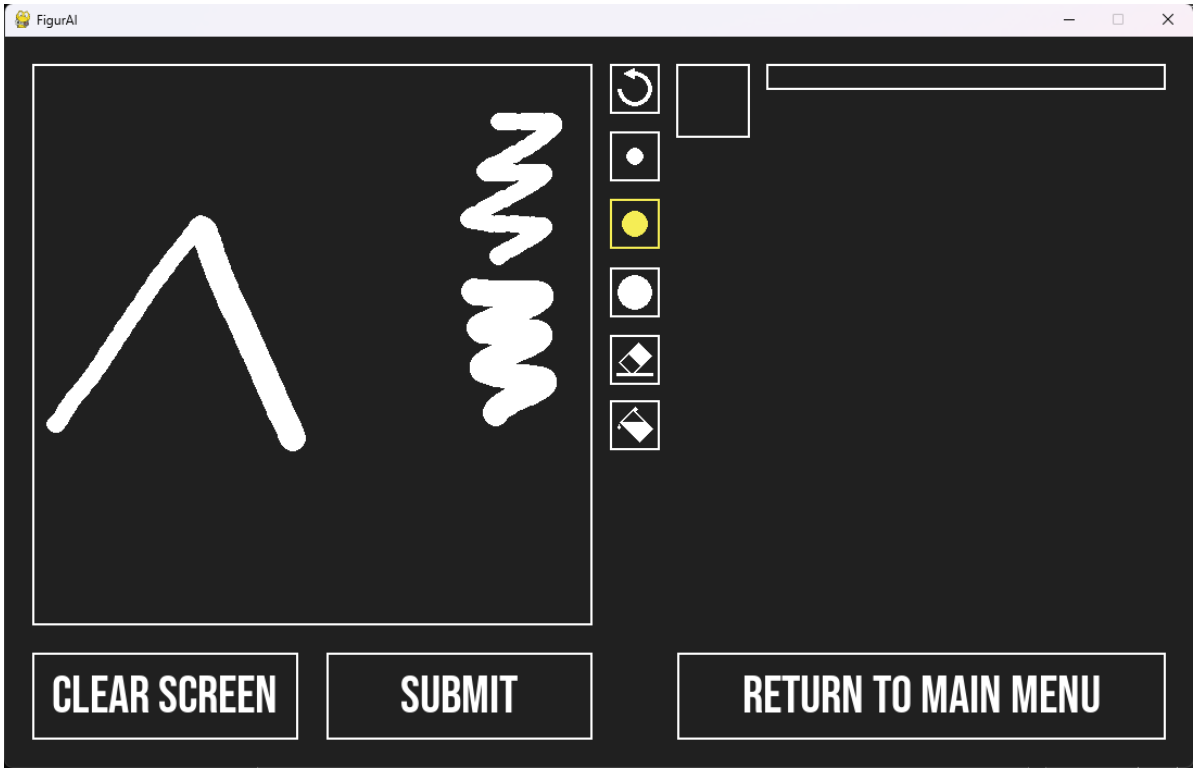
Test no.	Functional requirement being tested	Description of tests	Expected Results	Actual Results
1	Inputs on the training screen	a) Return to Main Menu	a) The user is returned to the main menu	All tests were successful
2	Inputs on the testing screen	a) Draw with small pen size selected b) Draw with medium pen size selected c) Draw with large pen size selected d) Test using the eraser tool e) Test using the fill tool f) Undo last 2 actions g) Submit drawing h) Clear the screen i) Return to Main Menu	a) Thin line drawn on the screen b) Medium line drawn c) Thick line drawn d) Drawing erased at mouse pointer e) Filled in the enclosed area around the mouse pointer f) The drawing Is reset to before using the eraser and the fill tool g) The program analyzes the shape and displays the results h) The drawing is deleted i) Return to main menu	All tests were successful
3	Inputs on the info screen	a) Return to Main Menu	a) The user is returned to the main menu	All tests were successful
4	Inputs on the settings screen	a) Reposition the save interval slider to 1 b) Reposition the save interval slider to 50 c) Reposition the save interval slider to 500 d) Reposition the rotation steps slider to 4 e) Reposition the rotation steps slider to 200 f) Reposition the rotation steps slider to 2000 g) Click the reset database button h) Click yes at the warning popup i) Click no at the warning popup j) Click the main menu button without saving k) Click yes at the warning popup l) Click no at the warning popup m) Click the apply button n) Click the main menu button after saving	a) The save interval setting is temporarily changed to 1 b) The save interval setting is temporarily changed to 50 c) The save interval setting is temporarily changed to 500 d) The rotation steps setting is temporarily changed to 4 e) The rotation steps setting is temporarily changed to 200 f) The rotation steps setting is temporarily changed to 2000 g) Warning popup appears h) All the weights are set to 0 and return to the settings screen i) Return to the settings screen j) Warning popup appears k) Return to main menu without applying temporary changes l) Return to settings screen m) Apply the temporary changes n) The user is returned to the main menu	All tests were successful

Functional requirement testing Evidence:

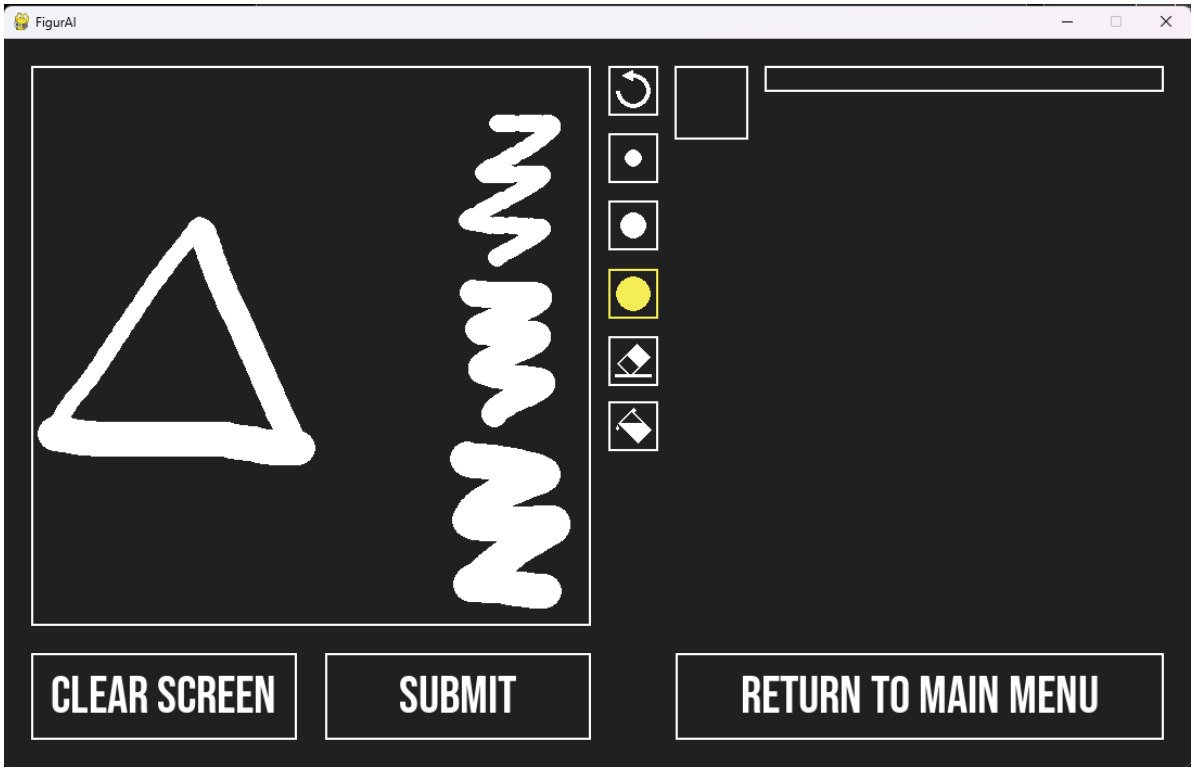
Test 2a:



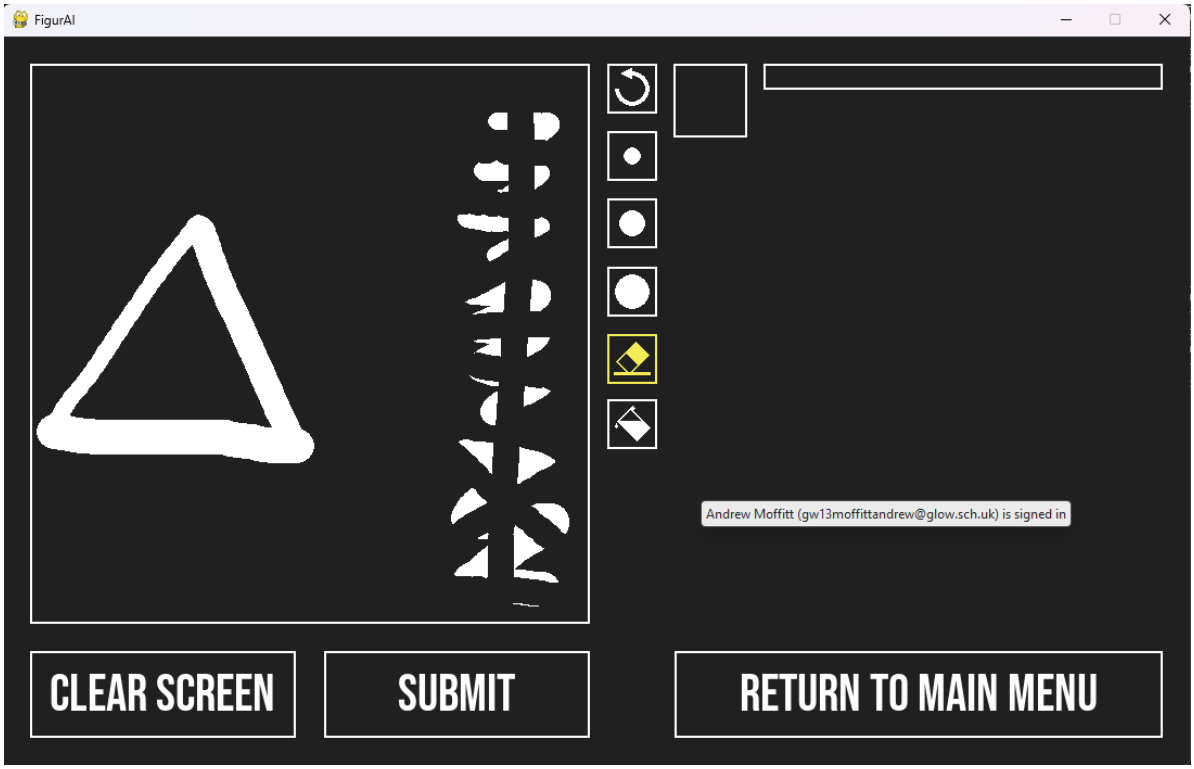
Test 2b:



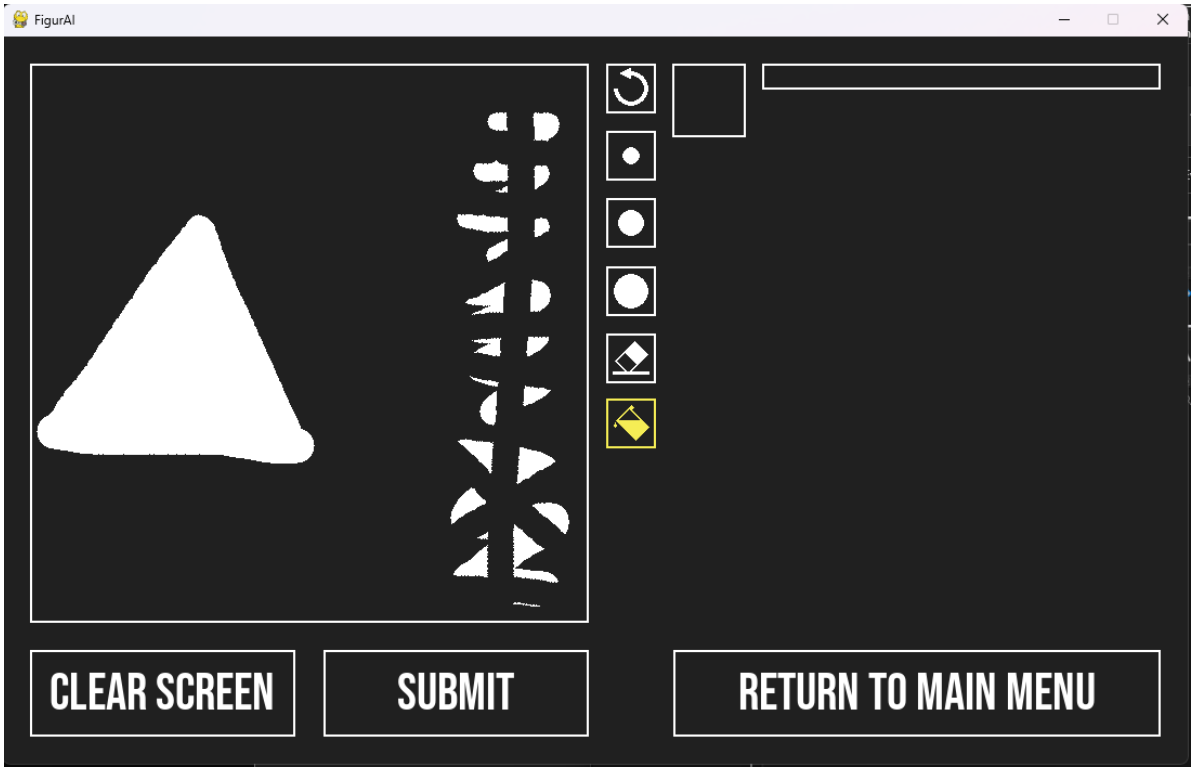
Test 2c:



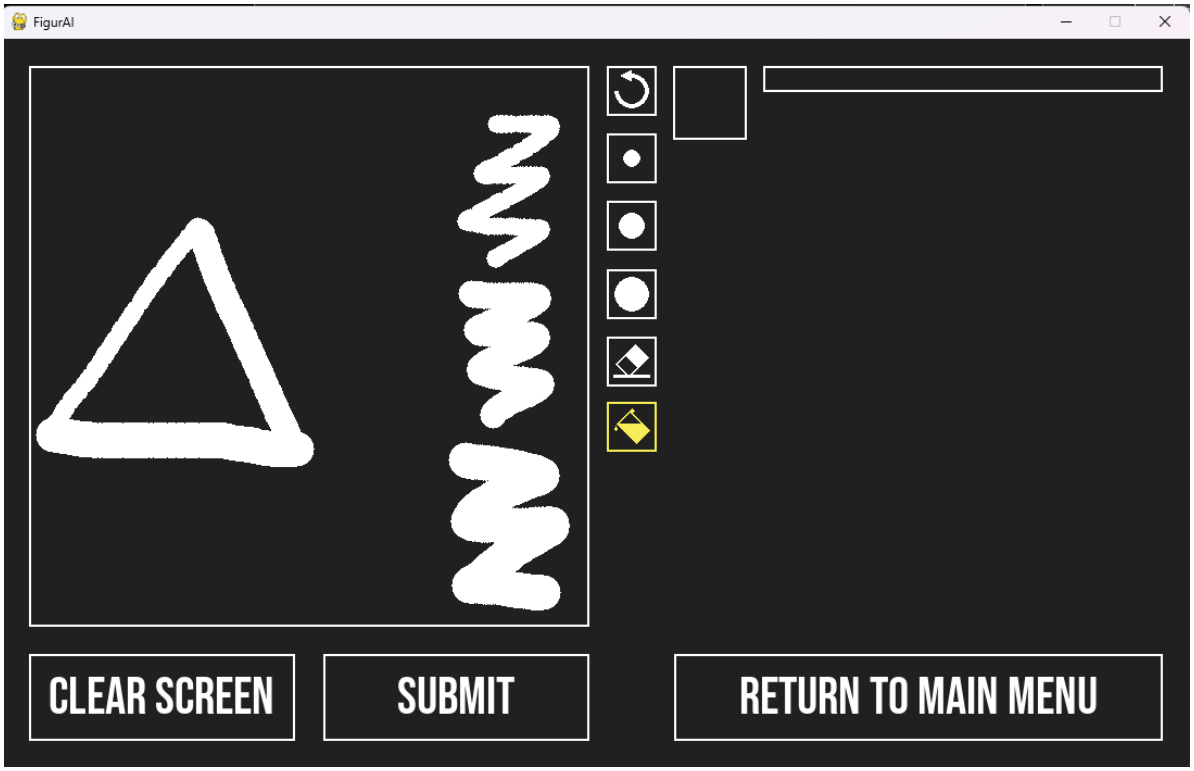
Test 2d:



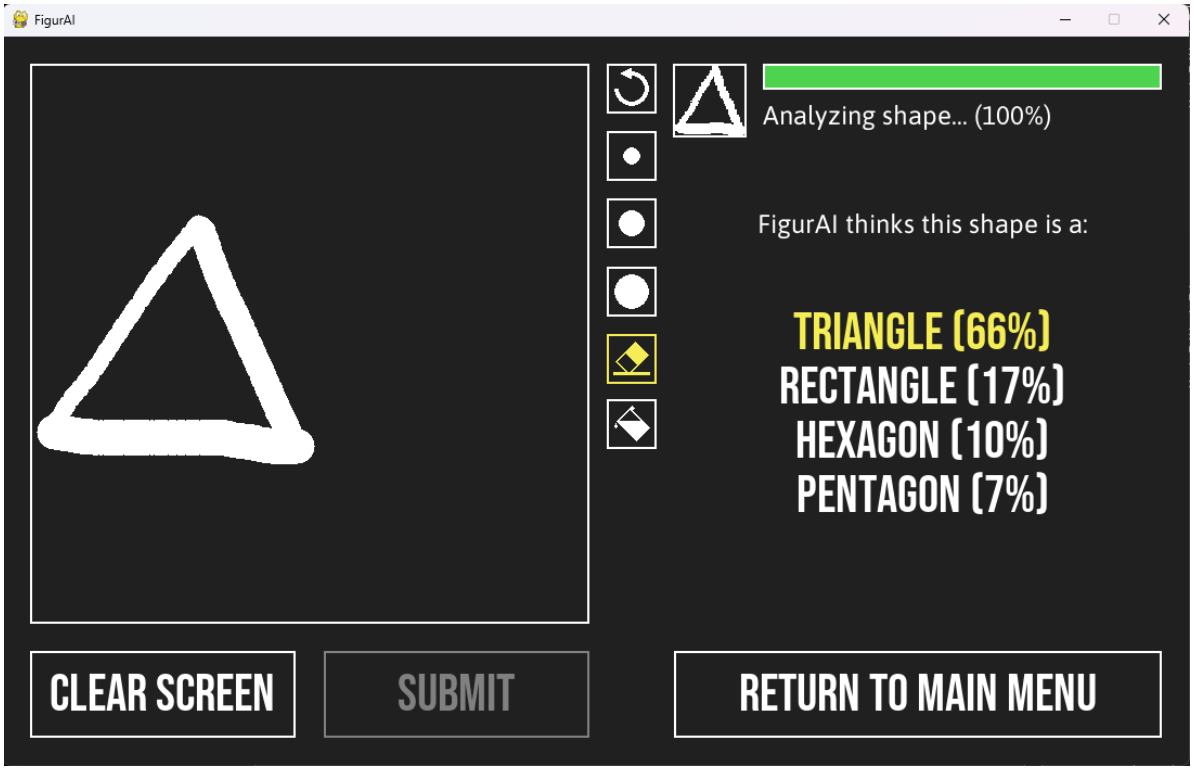
Test 2e:



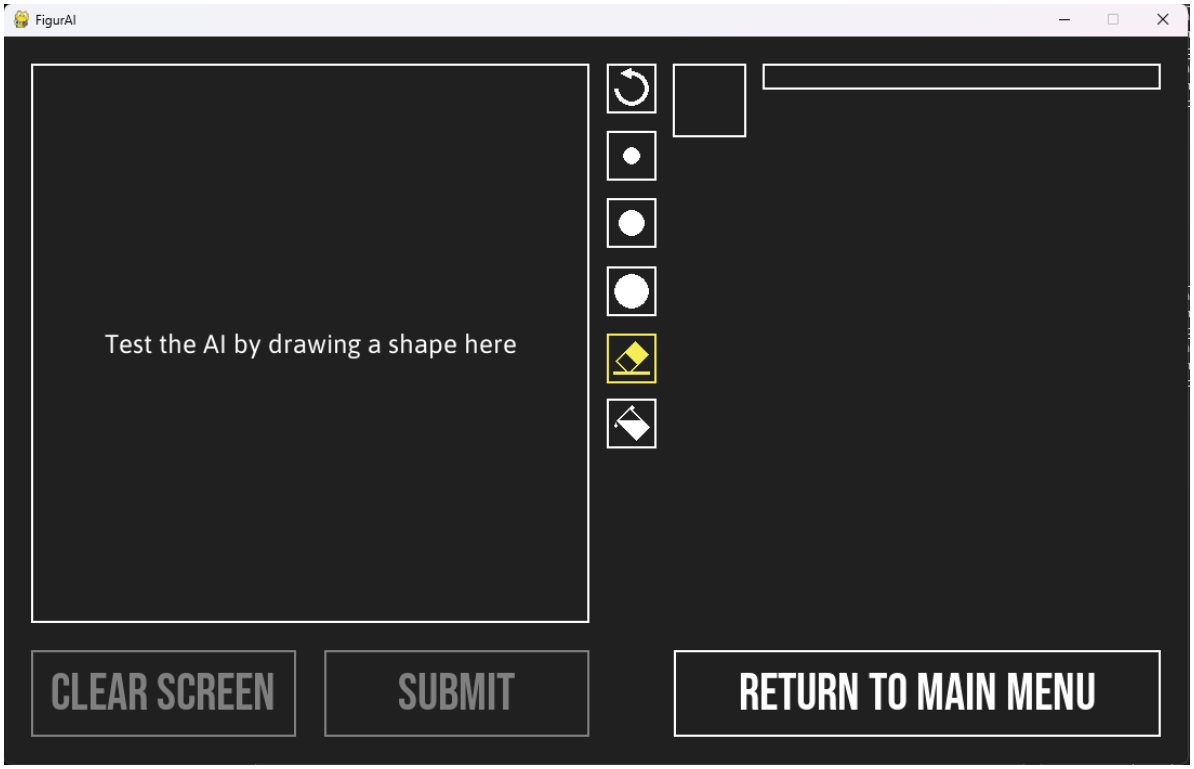
Test 2f:



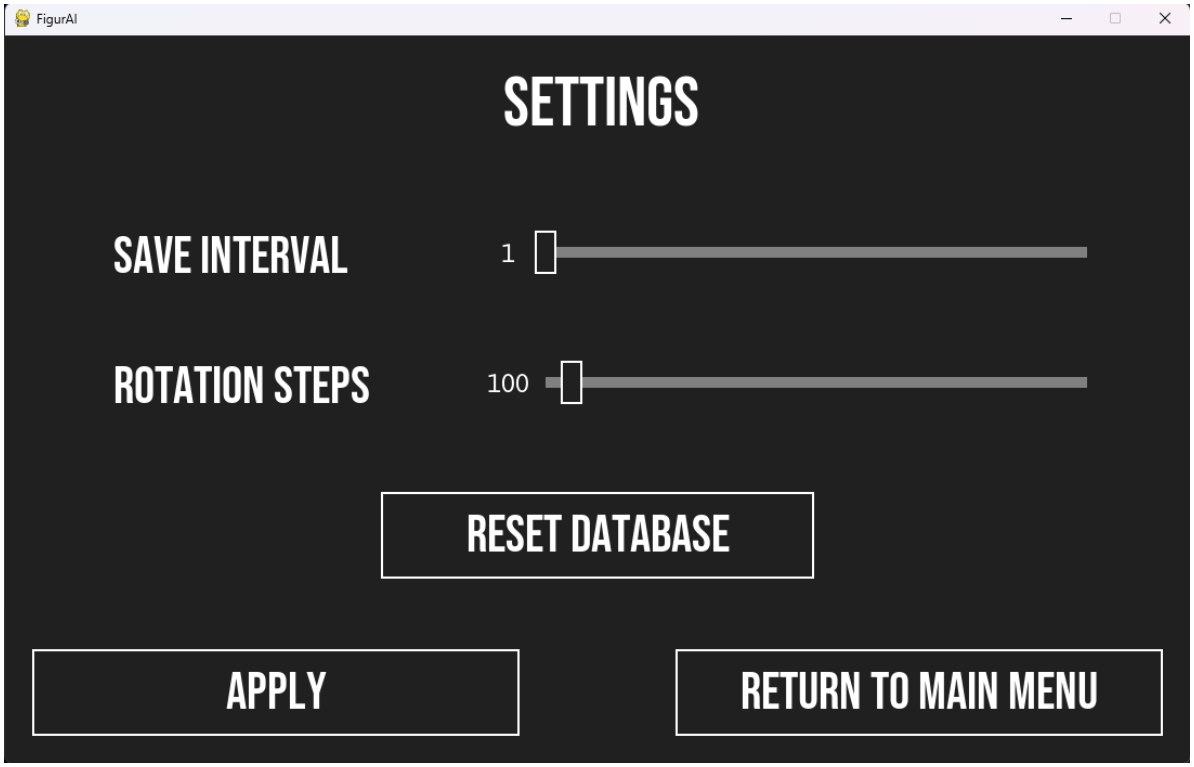
Test 2g:



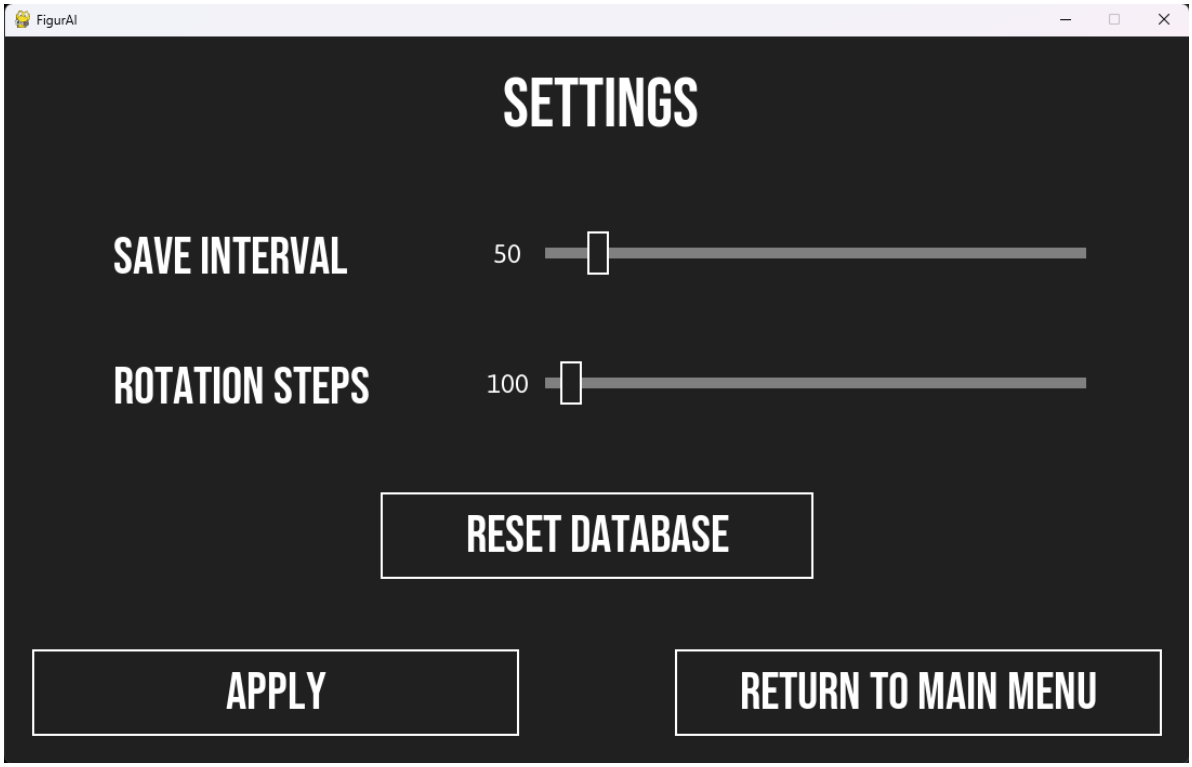
Test 2h:



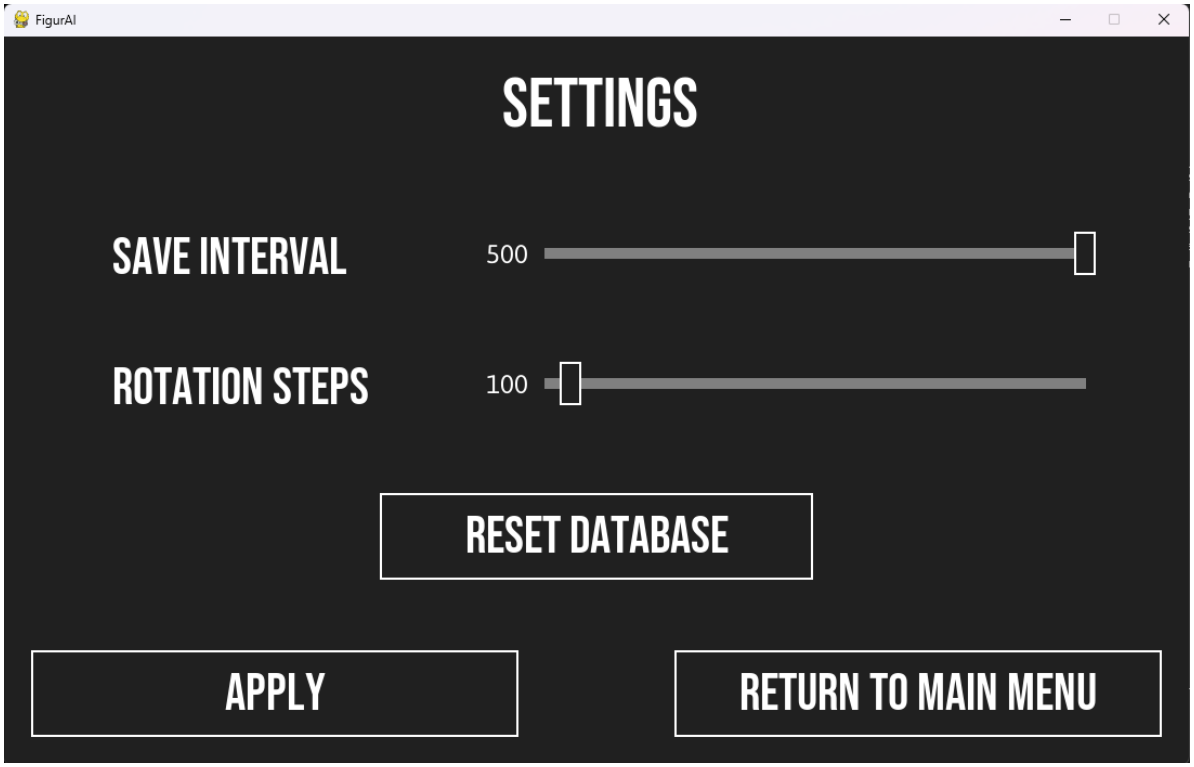
Test 4a:



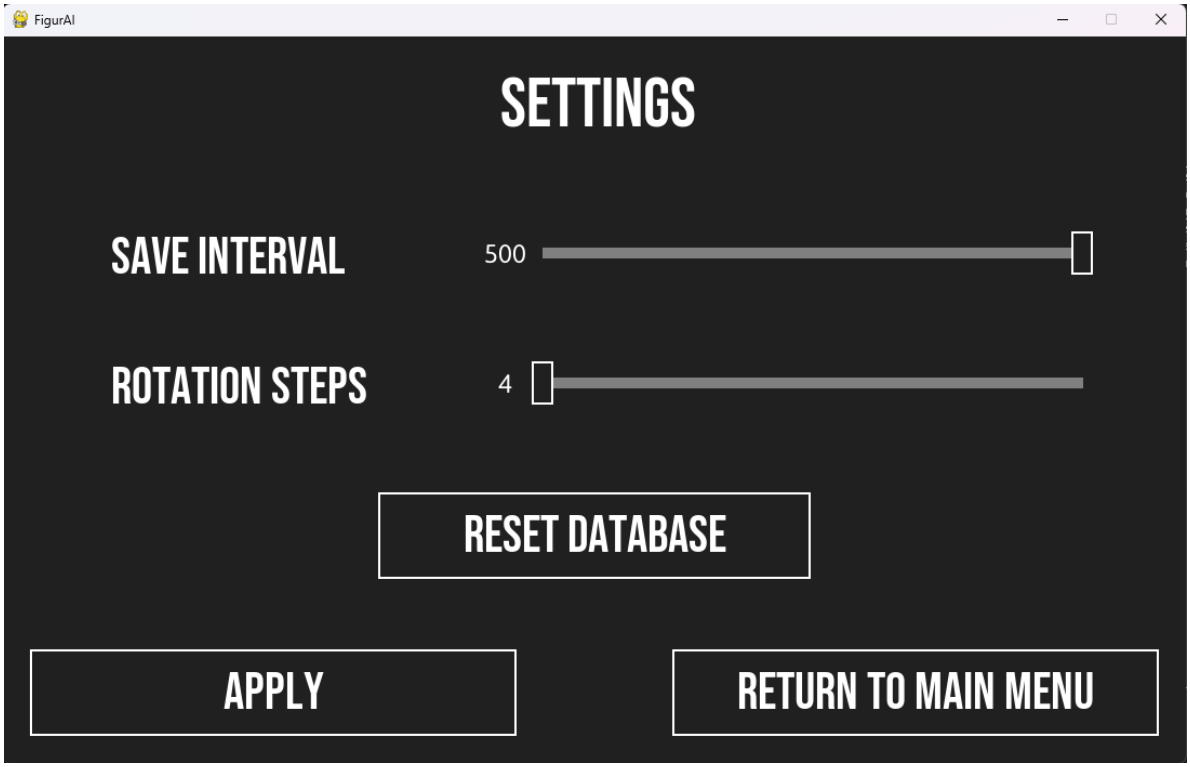
Test 4b:



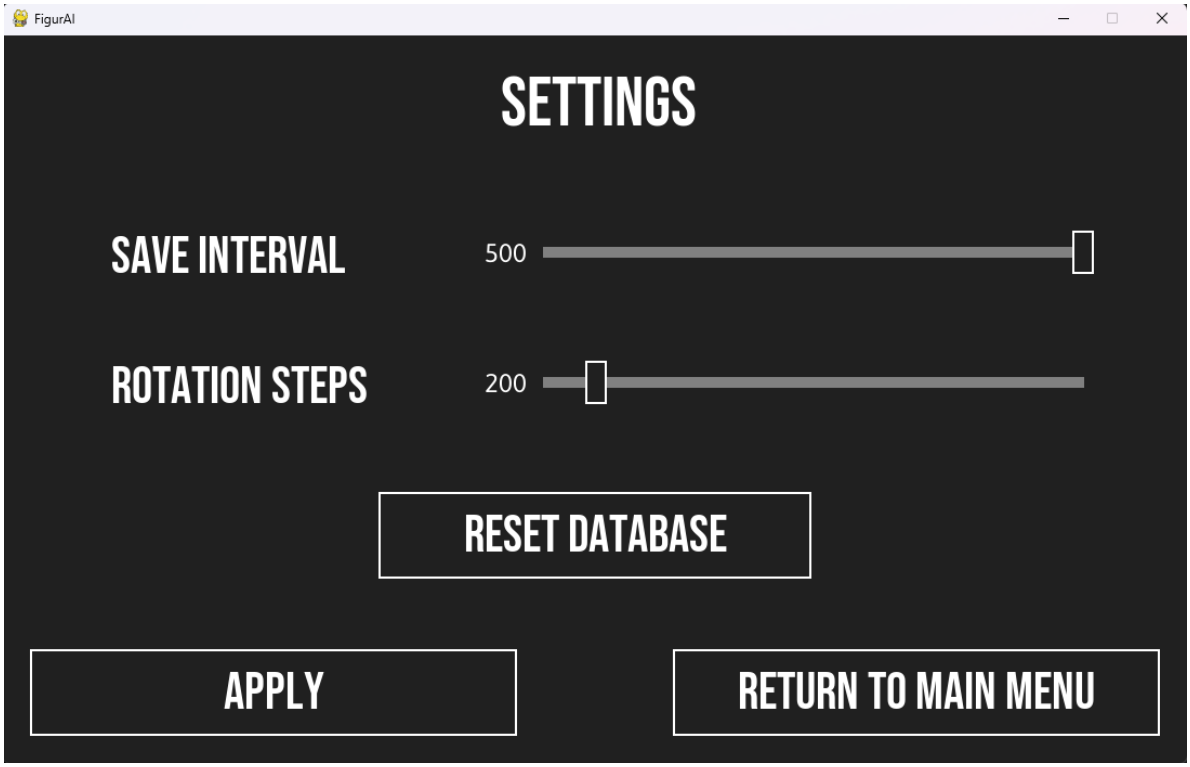
Test 4c:



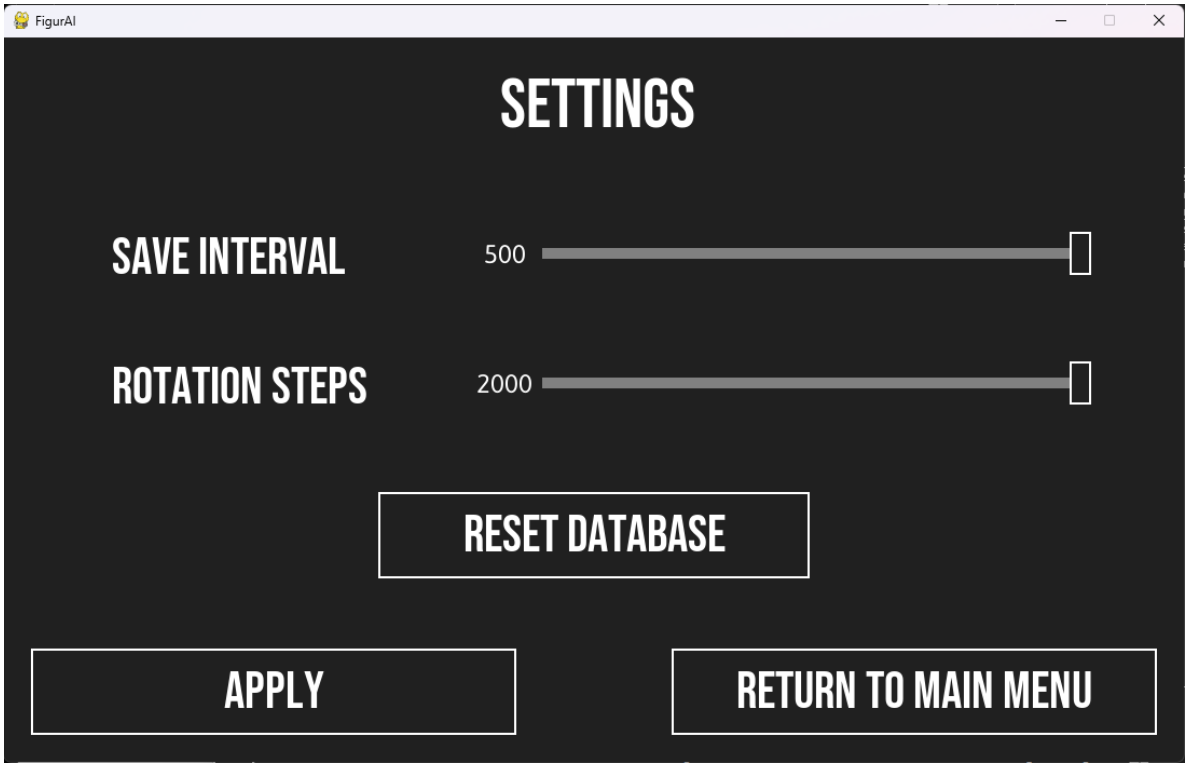
Test 4d:



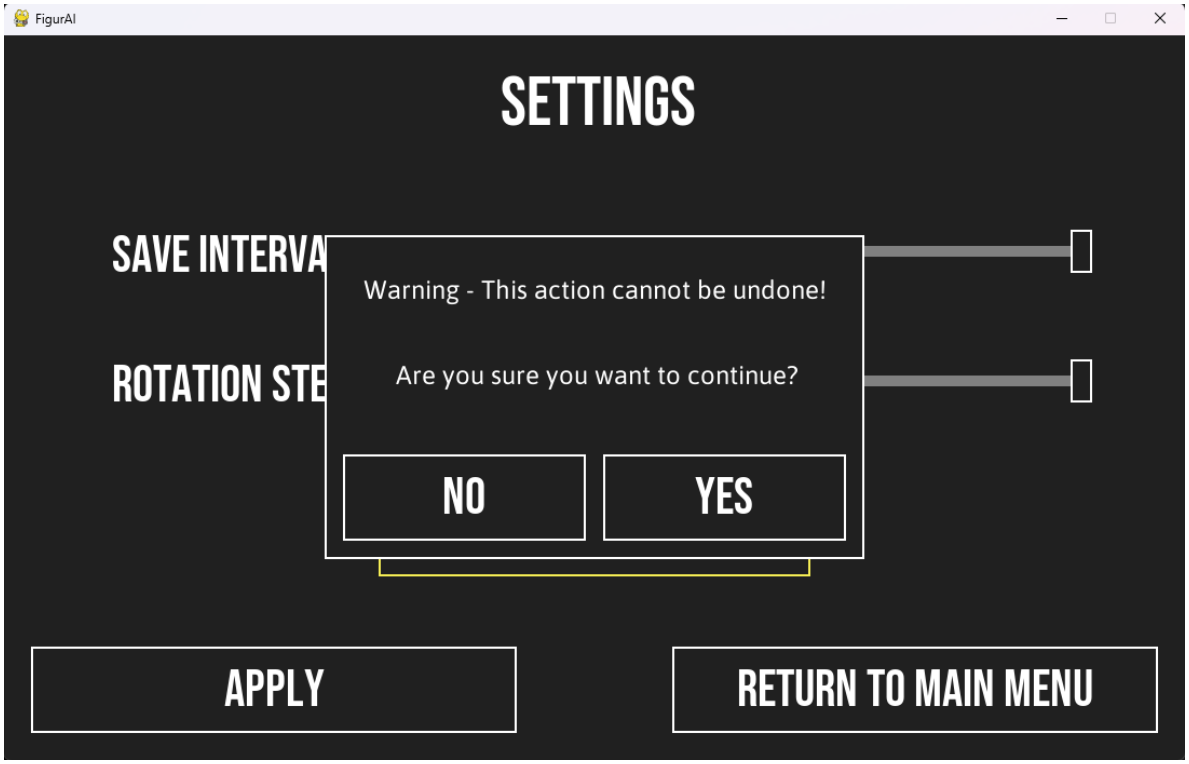
Test 4e:



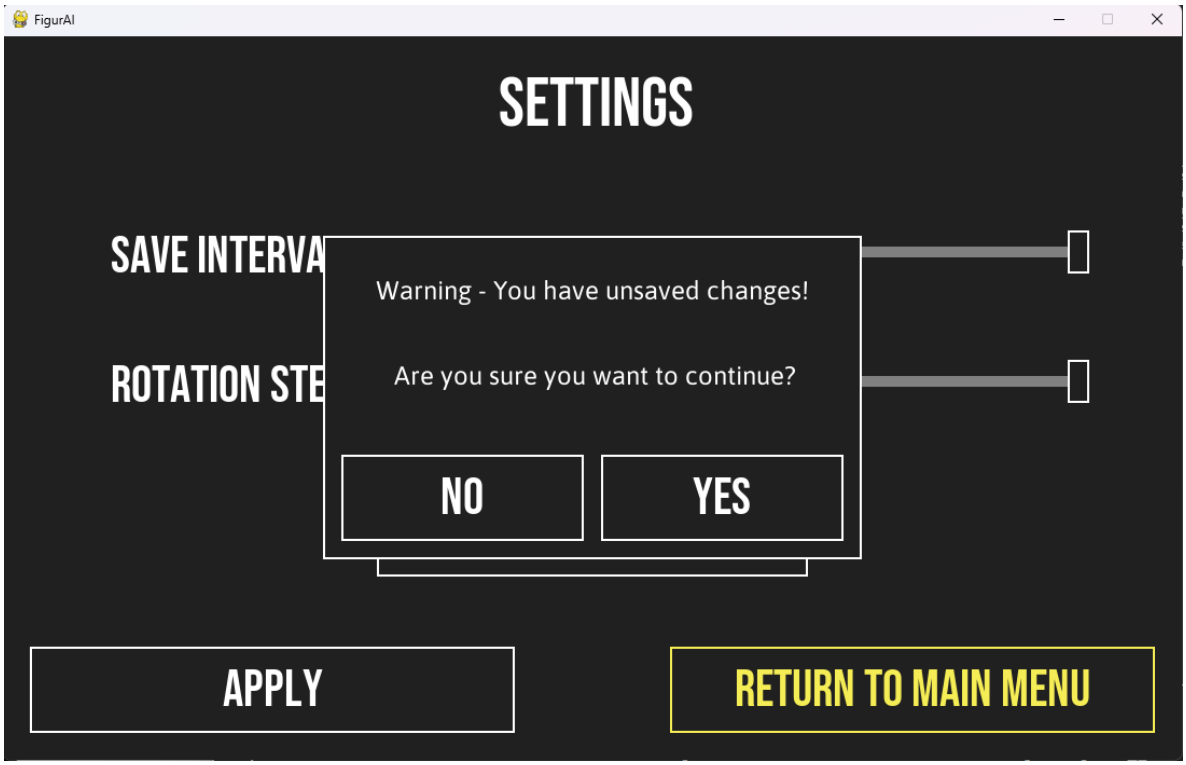
Test 4f:



Test 4g:



Test 4j:



Evaluation

Fitness for purpose:

For the most part, my program matches the requirements that I have stated in my requirements specification. One slight change I have made to the program that was not specified previously was adding loading screens, as loading some parts of my program took longer to load than I anticipated, and I wanted to add some informative text on the screen to let the user know that the program is loading. I also have made my AI position, scale, stretch and rotation invariant which I did not mention in my initial description of my project as I wasn't originally sure if I would be able to implement these features.

Robustness:

This program only takes in basic inputs like mouse button clicks, and all inputs have been tested in the functional requirements testing stage, so I do not believe it is possible to cause the program to crash or behave abnormally through invalid user input. One thing that might cause abnormal behaviour is if the program is forcibly closed (e.g. through task manager) whilst the weights are being saved to the database. I have not tested this situation, but it may cause the database to become corrupted and the program may not function properly when trying to access it again.

Maintainability:

Readability:

I have added lots of white space, separated code into coherent functions and used sensible variable names to my program to try to improve the readability, however the readability is still poor as I have added very little internal commentary. As I started coming close to the project deadline, I was short on time and had to prioritize my documentation over the readability of the program. If given more time to complete the project, I would go through my program code and add much more internal commentary.

Modularity:

My program is very modular as it makes efficient use of functions and classes, allowing changes to be made easily without having to modify sections of code multiple times. However, I started running out of time when I was working on the settings screen, so I focussed on making the sliders fully functional and didn't have time to focus on modularity, so this part of the program is not very modular. This is something I would try to improve if I had more time.

Appendix

Key:

Yellow text – Object Oriented Programming
Green text – Database Integration
Light Blue text – Insertion Sort Algorithm
Aqua text – Polymorphism
Grey text – Inheritance

FigurAI.py:

```
# FigurAI - An Artificial Intelligence Shape Recognition program by Andrew
Moffitt
# Generates random shapes, then trains itself using machine learning to figure
out what shapes got generated

from datetime import datetime
from init import *
from bitmap import *
from dbConn import *
from shapeGen import *

class Shape:

    def __init__(this, shapeName, shapeType, cornerNo):

        this.__shapeName = shapeName
        this.__shapeType = shapeType
        this.__cornerNo = cornerNo
        this.__bitmap = numpy.full((shapeRes, shapeRes), False, dtype=type)
        this.__weightmap = numpy.full((weightRes, weightRes), 0, dtype=int)
        this.__needsSave = False

    def getBitmap(this):

        return this.__bitmap

    def getWeights(this):

        return this.__weightmap

    def getNeedsSave(this):

        return this.__needsSave

    def setWeights(this, weightmap):

        this.__weightmap = weightmap

    def generate(this):

        shapeCheck = False
        while not shapeCheck:

            if this.__shapeName == "Circle":
```

```
                bitmap = genCircle(this.__shapeType)
            else:
                bitmap = genPolygon(this.__shapeType, this.__cornerNo)

            shapeCheck = validateBitmap(bitmap)
            if not shapeCheck:
                print("Invalid shape generated! Regenerating new shape...")

        this.__bitmap = bitmap

    def adjustWeights(this, bitmap, adjustValue):

        for y in range(weightRes):
            for x in range(weightRes):
                if bitmap[x, y]:
                    this.__weightmap[x, y] += adjustValue

        this.__needsSave = True

    def saveWeights(this):

        updateDB(this.__shapeType + this.__shapeName, this.__weightmap)
        this.__needsSave = False

# AI functions

def compareBitmapToWeights(bitmap, weightmap):

    total = 0
    for y in range(weightRes):
        for x in range(weightRes):
            if bitmap[x, y]:
                total += weightmap[x, y]

    return total

def guessShape(bitmap, buttons):

    croppedBitmap = cropBitmap(bitmap)

    bitmaps = []
    rotationSteps = int(globalSettings.getRotationSteps() / 4)
    progress = 0
    highestTotals = {}
    for shapeName in shapeList:
        highestTotals[shapeName] = 0

    finalRotation = False
    analyzing = True
    for rotationIndex in range(4):
        for rotation in range(rotationSteps):
            if analyzing:

                if rotationIndex == 0:
                    processedBitmap = cropBitmap(rotateBitmap(croppedBitmap,
rotation * (90 / rotationSteps)))
                    bitmaps.append(processedBitmap)
                else:
                    processedBitmap = rotateBitmap(bitmaps[rotation], 90 *
rotationIndex)
```

```
                for shapeName in shapeList:

                    mixedTotal = 0
                    for shapeType in shapeTypes:
                        total = compareBitmapToWeights(processedBitmap,
shapes[shapeType + shapeName].getWeights())
                        if total > 0:
                            mixedTotal += total

                    if mixedTotal > highestTotals[shapeName]:
                        highestTotals[shapeName] = mixedTotal

                displayBitmap(processedBitmap, Pos(620, 27))
                progress += 1 / (4 * rotationSteps)

                pyg.draw.rect(display, darkgrey, pyg.Rect((701, 58), (300,
30)))
                display.blit(smallFont.render("Analyzing shape... (" +
format(progress * 100, ".0f") + "%)", 1, white), (701, 58))

                pyg.draw.rect(display, green, pyg.Rect((703, 27),
(round(progress * 364, 0), 20)))

                if rotationIndex == 3 and rotation == rotationSteps - 1:
                    finalRotation = True
                displayResults(highestTotals, finalRotation)

                action, running = detectAction(buttons)
                if action != "":
                    analyzing = False

    return action, running

# Main pages

def mainMenu():

    display.fill(darkgrey)
    display.blit(titleFont.render("Figur", 1, white), (285, 60))
    display.blit(titleFont.render("AI", 1, yellow), (655, 60))
    buttons = [
        TextButton(Pos(132, 200), (400, 100), "exit_train", "Train AI",
bigFont),
        TextButton(Pos(562, 200), (400, 100), "exit_test", "Test AI", bigFont),
        TextButton(Pos(132, 330), (400, 100), "exit_info", "Info", bigFont),
        TextButton(Pos(562, 330), (400, 100), "exit_settings", "Settings",
bigFont),
        TextButton(Pos(347, 460), (400, 100), "exit", "Quit", bigFont)
    ]

    running = True
    while running:
        action, running = detectAction(buttons)

    if action == "train":
        textScreen("Loading...")
        trainingMode()
    elif action == "test":
        textScreen("Loading...")
```

```
        testingMode()
    elif action == "info":
        helpScreen()
    elif action == "settings":
        settings()

def trainingMode():

    display.fill(darkgrey)
    buttons = [
        TextButton(Pos(619, 566), (450, 80), "exit_menu", "Return to Main
Menu", font)
    ]

    results = {}
    for i in range(len(shapeList)):
        shapeName = shapeList[i]
        results[shapeName] = []
        textRect = smallFont.render(shapeName, 1, white)
        display.blit(textRect, textRect.get_rect(center = (i * 143 + 124, 40)))
        for i2 in range(len(shapeTypes)):
            if i == 0:
                textRect =
pyg.transform.rotate(smallFont.render(shapeTypes[i2][:-1], 1, white), 90)
                display.blit(textRect, textRect.get_rect(center = (40, i2 * 143
+ 124)))
            displayWeights(shapes[shapeTypes[i2] + shapeName].getWeights(),
Pos(i * 143 + 60, i2 * 143 + 60))

    pyg.draw.rect(display, white, pyg.Rect((797, 60), (260, 260)))
    pyg.draw.rect(display, darkgrey, pyg.Rect((799, 62), (256, 256)))
    randShapeName = ""
    randShapeType = ""
    randBitmap = numpy.full((shapeRes, shapeRes), False, dtype=bool)

    running = True
    while running:

        for loop in range(globalSettings.getsaveInterval()):

            pyg.draw.rect(display, darkgrey, pyg.Rect((799, 0), (260, 60)))
            textRect = smallFont.render(randShapeType[:-1] + " " +
randShapeName, 1, white)
            display.blit(textRect, textRect.get_rect(center = (927, 40)))
            displayBitmap(randBitmap, Pos(799, 62))

            randShapeName = shapeList[randint(0, len(shapeList) - 1)]
            randShapeType = shapeTypes[randint(0, len(shapeTypes) - 1)]

            accuracies = []
            totalAccuracy = 0
            for shapeName2 in shapeList:
                accuracy = calcAccuracy(results[shapeName2])
                totalAccuracy += accuracy
                accuracies.append((shapeName2, accuracy))

            accuracies = insertionSort(accuracies)
            accuracies.append(("TOTAL ACCURACY", totalAccuracy /
len(accuracies)))
            pyg.draw.rect(display, darkgrey, pyg.Rect((775, 340), (300, 200)))
```

```
            for i in range(len(accuracies)):
                yOffset = 340
                if i == len(accuracies) - 1:
                    yOffset += 20
                display.blit(smallFont.render(accuracies[i][0] + ": " +
format(accuracies[i][1], ".2f") + "%", 1, white), (780, i * 30 + yOffset))

            for shapeName in shapeList:
                for shapeType in shapeTypes:
                    if running:

                        shape = shapes[shapeType + shapeName]
                        shape.generate()
                        croppedBitmap = cropBitmap(shape.getBitmap())
                        rotatedBitmap = cropBitmap(rotateBitmap(croppedBitmap,
randint(0, 360)))

                        if shapeName == randShapeName and shapeType ==
randShapeType:
                            randBitmap = copyBitmap(shape.getBitmap())

                        if compareBitmapToWeights(croppedBitmap,
shape.getWeights()) <= 0:
                            shape.adjustWeights(croppedBitmap, 1)
                            results[shapeName].append(False)
                        else:
                            results[shapeName].append(True)

                        if len(results[shapeName]) > 10000:
                            del results[shapeName][0]

                        for otherShapeName in shapeList:
                            if otherShapeName != shapeName:
                                for otherShapeType in shapeTypes:
                                    if compareBitmapToWeights(rotatedBitmap,
shapes[otherShapeType + otherShapeName].getWeights()) > 0:
                                        shapes[otherShapeType +
otherShapeName].adjustWeights(rotatedBitmap, -1)
                                        results[otherShapeName].append(False)
                                    else:
                                        results[otherShapeName].append(True)

                                    if len(results[otherShapeName]) > 10000:
                                        del results[otherShapeName][0]

                        action, running = detectAction(buttons)

                        if action == "menu":
                            textScreen("Saving...")

        # Save and display the weight images for each shape

        if running:
            pyg.draw.rect(display, darkgrey, pyg.Rect((160, 500), (300, 100)))
            textRect = bigFont.render("Saving...", 1, white)
            display.blit(textRect, textRect.get_rect(center = (310, 550)))
            pyg.display.flip()

        for i in range(len(shapeList)):
            for i2 in range(len(shapeTypes)):
```

```
                shapeID = shapeTypes[i2] + shapeList[i]
                if shapes[shapeID].getNeedsSave():
                    displayWeights(shapes[shapeID].getWeights(), Pos(i * 143 +
60, i2 * 143 + 60))
                    shapes[shapeID].saveWeights()

        pyg.draw.rect(display, darkgrey, pyg.Rect((10, 500), (600, 170)))
        textRect = bigFont.render("Saved", 1, white)
        display.blit(textRect, textRect.get_rect(center = (310, 550)))
        textRect = font.render("Last saved at " +
datetime.now().strftime("%H:%M:%S"), 1, white)
        display.blit(textRect, textRect.get_rect(center = (310, 620)))

    if action == "menu":
        mainMenu()

def testingMode():

    display.fill(darkgrey)
    buttons = [
        TextButton(Pos(619, 566), (450, 80), "exit_menu", "Return to Main
Menu", font),
        BrushButton(Pos(557, 25), (46, 46), "undo"),
        BrushButton(Pos(557, 87), (46, 46), "brush_8"),
        BrushButton(Pos(557, 149), (46, 46), "brush_12"),
        BrushButton(Pos(557, 212), (46, 46), "brush_16"),
        BrushButton(Pos(557, 274), (46, 46), "brush_eraser"),
        BrushButton(Pos(557, 334), (46, 46), "brush_fill")
    ]

    clearButton = TextButton(Pos(25, 566), (245, 80), "clear", "Clear Screen",
font)
    submitButton = TextButton(Pos(296, 566), (245, 80), "submit", "Submit",
font)

    brush = 12
    buttons[3].setPermColor(yellow)
    bitmapSize = 512

    # Drawing box outline
    pyg.draw.rect(display, white, pyg.Rect((25, 25), (bitmapSize + 4,
bitmapSize + 4)))

    # Progress bar outline
    pyg.draw.rect(display, white, pyg.Rect((701, 25), (368, 24)))

    # Mini shape display outline
    pyg.draw.rect(display, white, pyg.Rect((618, 25), (68, 68)))

    running = True
    while running:

        # Reset drawing box
        pyg.draw.rect(display, darkgrey, pyg.Rect((27, 27), (bitmapSize,
bitmapSize)))

        # Show display text
        textRect = smallFont.render("Test the AI by drawing a shape here", 1,
white)
        display.blit(textRect, textRect.get_rect(center = (283, 283)))
```

```
        emptyBitmap = True

        # Hide the "Analyzing..." text
        pyg.draw.rect(display, darkgrey, pyg.Rect((701, 58), (300, 30)))

        # Reset mini shape display
        pyg.draw.rect(display, darkgrey, pyg.Rect((620, 27), (64, 64)))

        # Reset progress bar
        pyg.draw.rect(display, darkgrey, pyg.Rect((703, 27), (364, 20)))

        # Reset shape guesses
        pyg.draw.rect(display, darkgrey, pyg.Rect((618, 108), (476, 558)))

        # Reset "Clear Screen" and "Submit" buttons
        buttons = buttons[:7]
        clearButton.setColor(lightgrey)
        submitButton.setColor(lightgrey)

        # Reset the bitmap and the bitmap history
        bitmap = numpy.full((bitmapSize, bitmapSize), False, dtype=bool)
        bitmapBackups = []

        mouseDown = False
        fillDown = False
        clearScreen = False
        while running and not clearScreen:

            mousePos = pyg.mouse.get_pos()
            if pyg.mouse.get_pressed()[0] and mousePos[0] in range(27,
bitmapSize + 27) and mousePos[1] in range(27, bitmapSize + 27):

                if emptyBitmap and brush != "eraser":
                    pyg.draw.rect(display, darkgrey, pyg.Rect((27, 27),
(bitmapSize, bitmapSize)))
                    emptyBitmap = False

                mouseDown = True
                bitmapPos = Pos(mousePos[0] - 27, mousePos[1] - 27)
                if brush == "fill":

                    if not bitmap[bitmapPos.x, bitmapPos.y]:
                        fillDown = True
                        fillShape(bitmap, bitmapPos, 1)
                        pyg.draw.rect(display, darkgrey, pyg.Rect((27, 27),
(bitmapSize, bitmapSize)))
                        for y in range(bitmapSize):
                            for x in range(bitmapSize):
                                if bitmap[x, y]:
                                    pyg.draw.rect(display, white, pyg.Rect((x +
27, y + 27), (1, 1)))
                    else:
                        mouseDown = False

                elif brush == "eraser":

                    if not emptyBitmap:
                        pyg.draw.circle(display, darkgrey, mousePos, 12)
                        drawDot(bitmap, bitmapPos, 12, True)
```

```
                        if bitmapPos.x not in range(12, bitmapSize - 12):
                            pyg.draw.rect(display, white, pyg.Rect((25, 25),
(2, bitmapSize + 4)))
                            pyg.draw.rect(display, white, pyg.Rect((539, 25),
(2, bitmapSize + 4)))
                        if bitmapPos.y not in range(12, bitmapSize - 12):
                            pyg.draw.rect(display, white, pyg.Rect((25, 25),
(bitmapSize + 4, 2)))
                            pyg.draw.rect(display, white, pyg.Rect((25, 539),
(bitmapSize + 4, 2)))
                    else:
                        mouseDown = False

                else:
                    pyg.draw.circle(display, white, mousePos, brush)
                    drawDot(bitmap, bitmapPos, brush)

                    if bitmapPos.x not in range(brush, bitmapSize - brush):
                        pyg.draw.rect(display, darkgrey, pyg.Rect((10, 10),
(15, 546)))
                        pyg.draw.rect(display, darkgrey, pyg.Rect((541, 10),
(15, 546)))
                    if bitmapPos.y not in range(brush, bitmapSize - brush):
                        pyg.draw.rect(display, darkgrey, pyg.Rect((10, 10),
(546, 15)))
                        pyg.draw.rect(display, darkgrey, pyg.Rect((10, 541),
(546, 15)))

            elif mouseDown or fillDown:

                if len(buttons) == 7:
                    buttons.append(clearButton)
                if len(buttons) == 8:
                    buttons.append(submitButton)

                bitmapBackups.append(copyBitmap(bitmap))
                if len(bitmapBackups) > 50:
                    del bitmapBackups[0]
                mouseDown = False
                fillDown = False

            action, running = detectAction(buttons)
            if action == "submit":

                # Disable the "Submit" button
                buttons.remove(submitButton)
                submitButton.setColor(lightgrey)

                # Reset progress bar
                pyg.draw.rect(display, darkgrey, pyg.Rect((703, 27), (364,
20)))

                # Reset shape guesses
                pyg.draw.rect(display, darkgrey, pyg.Rect((618, 108), (476,
558)))

                # Show text
                display.blit(smallFont.render("FigurAI thinks this shape is
a:", 1, white), (696, 158))
```

```
                action, running = guessShape(bitmap, [buttons[0], clearButton])
                if action == "clear":
                    clearScreen = True

            if action == "clear":
                clearScreen = True
            elif action == "undo":
                if len(bitmapBackups) > 1:
                    bitmap = copyBitmap(bitmapBackups[-2])
                    del bitmapBackups[-2:]
                    pyg.draw.rect(display, darkgrey, pyg.Rect((27, 27),
(bitmapSize, bitmapSize)))
                    for y in range(bitmapSize):
                        for x in range(bitmapSize):
                            if bitmap[x, y]:
                                pyg.draw.rect(display, white, pyg.Rect((x + 27,
y + 27), (1, 1)))
                    mouseDown = True

                else:
                    clearScreen = True
            elif action[:5] == "brush":
                for i in range(2, 7):
                    if buttons[i].getAction() == action:
                        buttons[i].setPermColor(yellow)
                    else:
                        buttons[i].setPermColor(white)

                brush = action[6:]

                if brush != "eraser" and brush != "fill":
                    brush = int(action[6:])

    if action == "menu":
        mainMenu()

def helpScreen():

    display.fill(darkgrey)
    buttons = [
        TextButton(Pos(322, 566), (450, 80), "exit_menu", "Return to Main
Menu", font)
    ]

    display.blit(bigFont.render("Info", 1, white), (505, 25))
    display.blit(smallFont.render("FigurAI - A shape recognition program", 1,
white), (345, 120))
    display.blit(smallFont.render("This program allows you to train an AI using
randomly generated shapes", 1, white), (165, 165))
    display.blit(smallFont.render("to improve the accuracy, then you can test
the AI by drawing a shape.", 1, white), (180, 210))
    display.blit(smallFont.render("The AI will learn to recognize Circles,
Triangles, Rectangles, Pentagons and Hexagons.", 1, white), (90, 255))
    display.blit(smallFont.render("Settings Help:", 1, white), (475, 340))
    display.blit(smallFont.render("Save Interval - Sets the time interval
between automatic saves in training mode", 1, white), (125, 385))
    display.blit(smallFont.render("Rotation Steps - Sets the number of times
the user-drawn shape will rotate in testing mode", 1, white), (70, 430))
    display.blit(smallFont.render("Reset Database - Resets the training
progress for all the shapes", 1, white), (210, 475))
```

```
    running = True
    while running:
        action, running = detectAction(buttons)

    if action == "menu":
        mainMenu()

def settings():

    buttons = [
        TextButton(Pos(619, 566), (450, 80), "exit_menu", "Return to Main
Menu", font),
        globalSettings.getButton(0),
        globalSettings.getButton(1),
        TextButton(Pos(347, 421), (400, 80), "reset", "Reset Database", font)
    ]

    applyButton = TextButton(Pos(25, 566), (450, 80), "apply", "Apply", font)

    rotationSteps = globalSettings.getRotationSteps()
    saveInterval = globalSettings.getsaveInterval()

    buttons[1].setPos(Pos(saveInterval + 488, 180))
    buttons[2].setPos(Pos(int((rotationSteps - 1) / 4) + 489, 300))

    resetScreen = True
    pressedButton = None
    running = True
    while running:

        if resetScreen:

            display.fill(darkgrey)
            display.blit(bigFont.render("Settings", 1, white), (460, 25))
            display.blit(font.render("Save Interval", 1, white), (100, 175))
            display.blit(font.render("Rotation Steps", 1, white), (100, 295))

            pyg.draw.rect(display, lightgrey, pyg.Rect((499, 195), (500, 10)))
            pyg.draw.rect(display, lightgrey, pyg.Rect((499, 315), (500, 10)))

            textRect = smallFont.render(str(saveInterval), 1, white)
            display.blit(textRect, textRect.get_rect(center = (464, 200)))
            textRect = smallFont.render(str(rotationSteps), 1, white)
            display.blit(textRect, textRect.get_rect(center = (464, 320)))

            applyButton.setColor(lightgrey)

            resetScreen = False

        mousePos = pyg.mouse.get_pos()
        if not pyg.mouse.get_pressed()[0]:
            if pressedButton:
                if len(buttons) == 4:
                    buttons.append(applyButton)
                pressedButton = None

        action, running = detectAction(buttons)

        if action == "apply":
```

```
            globalSettings.setRotationSteps(rotationSteps)
            globalSettings.setsaveInterval(saveInterval)
            applyButton.setColor(lightgrey)
            buttons.remove(applyButton)

        for button in buttons[1:3]:
            if action == button.getAction():
                pressedButton = button
                xOffset = mousePos[0] - pressedButton.getPos().x

        if pressedButton:
            pyg.draw.rect(display, darkgrey, pyg.Rect((489,
pressedButton.getPos().y), (520, 40)))

            sliderValue = pressedButton.getPos().x - 489
            if pressedButton.getAction() == "set_saveInterval":
                saveInterval = sliderValue + 1
                pyg.draw.rect(display, lightgrey, pyg.Rect((499, 195), (500,
10)))
                pyg.draw.rect(display, darkgrey, pyg.Rect((439, 185), (50,
30)))
                textRect = smallFont.render(str(saveInterval), 1, white)
                display.blit(textRect, textRect.get_rect(center = (464, 200)))

            if pressedButton.getAction() == "set_rotationSteps":
                rotationSteps = (sliderValue + 1) * 4
                pyg.draw.rect(display, lightgrey, pyg.Rect((499, 315), (500,
10)))
                pyg.draw.rect(display, darkgrey, pyg.Rect((439, 305), (50,
30)))
                textRect = smallFont.render(str(rotationSteps), 1, white)
                display.blit(textRect, textRect.get_rect(center = (464, 320)))

            newPos = Pos(mousePos[0] - xOffset, pressedButton.getPos().y)
            if newPos.x < 489:
                newPos.x = 489
            if newPos.x > 988:
                newPos.x = 988
            pressedButton.setPos(newPos)

        if action == "reset":
            resetScreen = True
            subAction = warningScreen("Warning - This action cannot be
undone!")
            if subAction == "continue":

                textScreen("Resetting weights...")

                for shapeName in shapeList:
                    for shapeType in shapeTypes:
                        shapeID = shapeType + shapeName
                        shapes[shapeID].setWeights(numpy.full((weightRes,
weightRes), 0, dtype=int))
                        shapes[shapeID].saveWeights()

            elif subAction == "quit":
                running = False

        if action == "menu":
            if len(buttons) == 5:
```

```
                subAction = warningScreen("Warning - You have unsaved
changes!")
                if subAction == "continue":
                    mainMenu()
                elif subAction == "cancel":
                    running = True
                    resetScreen = True
            else:
                mainMenu()

def textScreen(text):

    display.fill(darkgrey)
    textRect = bigFont.render(text, 1, white)
    display.blit(textRect, textRect.get_rect(center = (int(windowSize[0] / 2),
int(windowSize[1] / 2))))
    pyg.display.flip()

def warningScreen(text):

    buttons = [
        TextButton(Pos(314, 388), (225, 80), "exit_cancel", "No", font),
        TextButton(Pos(555, 388), (225, 80), "exit_continue", "Yes", font)
    ]
    pyg.draw.rect(display, white, pyg.Rect((297, 185), (500, 300)))
    pyg.draw.rect(display, darkgrey, pyg.Rect((299, 187), (496, 296)))
    textRect = smallFont.render(text, 1, white)
    display.blit(textRect, textRect.get_rect(center = (547, 235)))
    display.blit(smallFont.render("Are you sure you want to continue?", 1,
white), (363, 300))

    running = True
    while running:
        action, running = detectAction(buttons)

    return action


# Other functions

def detectAction(buttons):

    mousePos = pyg.mouse.get_pos()
    for button in buttons:
        if button.detectHover(mousePos):
            button.setColor(yellow)
        else:
            button.resetColor()

    action = ""
    for event in pyg.event.get():
        if event.type == pyg.MOUSEBUTTONDOWN and event.button == 1:
            for button in buttons:
                if button.detectHover(event.pos):
                    action = button.getAction()

        # Quit button
        if event.type == pyg.QUIT:
            action = "exit_quit"
```

```
    if action[:4] == "exit":
        action = action[5:]
        running = False
    else:
        running = True

    pyg.display.flip()

    return action, running

def insertionSort(array):

    for i in range(1, len(array)):
        currentValue = array[i]
        pos = i
        while pos > 0 and array[pos - 1][1] < currentValue[1]:
            array[pos] = array[pos - 1]
            pos -= 1
        array[pos] = currentValue

    return array

def displayWeights(weightmap, offsetPos):

    maxWeight = weightmap[0, 0]
    minWeight = weightmap[0, 0]
    for y in range(weightRes):
        for x in range(weightRes):
            if weightmap[x, y] > maxWeight:
                maxWeight = weightmap[x, y]
            if weightmap[x, y] < minWeight:
                minWeight = weightmap[x, y]

    brightnessScale = maxWeight - minWeight
    for y in range(weightRes):
        for x in range(weightRes):
            weightDiff = weightmap[x, y] - minWeight
            if weightDiff != 0:
                brightness = int(weightDiff / brightnessScale * 255)
            else:
                brightness = 0
            pyg.draw.rect(display, (brightness, brightness, brightness),
pyg.Rect((x * 2 + offsetPos.x, y * 2 + offsetPos.y), (2, 2)))

def displayBitmap(bitmap, offsetPos):

    bitmapRes = len(bitmap)
    pyg.draw.rect(display, darkgrey, (offsetPos.x, offsetPos.y, bitmapRes,
bitmapRes))

    for x in range(bitmapRes):
        for y in range(bitmapRes):
            if bitmap[x, y]:
                pyg.draw.rect(display, white, (x + offsetPos.x, y +
offsetPos.y, 1, 1))

def displayResults(totals, finalRotation):

    pyg.draw.rect(display, darkgrey, pyg.Rect((618, 198), (486, 368)))
    maxTotal = 5000
```

```
    results = []
    for shapeName in shapeList:
        total = totals[shapeName]
        if total > maxTotal / 100:
            if total > maxTotal:
                total = maxTotal
            results.append((shapeName, total))

    if len(results) == 0:
        results = [("idk bro", 0)]
    else:
        results = insertionSort(results)

    for i in range(len(results)):
        if i == 0 and finalRotation:
            color = yellow
        else:
            color = white
        yOffset = int((i - len(results) / 2) * 50)
        textRect = font.render(results[i][0] + " (" + format(results[i][1] /
maxTotal * 100, ".0f") + "%)", 1, color)
        display.blit(textRect, textRect.get_rect(center = (848, 373 +
yOffset)))

def calcAccuracy(results):

    if len(results) > 0:
        total = 0
        for result in results:
            total += result
        accuracy = total / len(results) * 100
    else:
        accuracy = 0

    return accuracy



### Main Program

print("\nLaunching FigurAI...")
textScreen("Launching FigurAI...")

print("\nRetrieving weights from '" + dbFileName + "' database...")
shapes = {}
for i in range(len(shapeList)):
    shapeName = shapeList[i]
    for shapeType in shapeTypes:
        shapeID = shapeType + shapeName
        shapes[shapeID] = Shape(shapeName, shapeType, i + 2)
        shapes[shapeID].setWeights(readWeights(shapeID))
print("Successfully retrieved all weights")

mainMenu()
print("\nExiting...")
textScreen("Exiting...")
pyg.quit()
```

init.py:

```
import pygame as pyg
import numpy
pyg.init()

# Class definitions

class Pos:

    def __init__(this, x, y):

        this.x = x
        this.y = y

class Button:

    def __init__(this, pos, dimensions, action):

        this.__pos = pos
        this.__dimensions = dimensions
        this.__action = action
        this.__color = white

    def draw(this):

        pyg.draw.rect(display, this.__color, pyg.Rect((this.__pos.x,
this.__pos.y), this.__dimensions))
        pyg.draw.rect(display, darkgrey, pyg.Rect((this.__pos.x + 2,
this.__pos.y + 2), (this.__dimensions[0] - 4, this.__dimensions[1] - 4)))

    def detectHover(this, mousePos):

        if mousePos[0] in range(this.__pos.x, this.__pos.x +
this.__dimensions[0]) and mousePos[1] in range(this.__pos.y, this.__pos.y +
this.__dimensions[1]):
            hover = True
        else:
            hover = False

        return hover

    def getPos(this):

        return this.__pos

    def getDimensions(this, index):

        return this.__dimensions[index]

    def getAction(this):

        return this.__action

    def getColor(this):

        return this.__color

    def setPos(this, pos):
```

```
        this.__pos = pos

    def setColor(this, color):

        this.__color = color
        this.draw()

    def resetColor(this):

        this.setColor(white)

class TextButton(Button):

    def __init__(this, pos, dimensions, action, text, buttonFont):
        super().__init__(pos, dimensions, action)
        this.__text = text
        this.__font = buttonFont
        this.draw()

    def draw(this):

        super().draw()
        buttonTextRect = this.__font.render(this.__text, 1, this.getColor())
        display.blit(buttonTextRect, buttonTextRect.get_rect(center =
(this.getPos().x + this.getDimensions(0) / 2, this.getPos().y +
this.getDimensions(1) / 2)))

class BrushButton(Button):

    def __init__(this, pos, dimensions, action):
        super().__init__(pos, dimensions, action)
        this.__whiteImg = pyg.image.load(assetFolder + action + "_white.png")
        this.__yellowImg = pyg.image.load(assetFolder + action + "_yellow.png")
        this.__permColor = white
        this.draw()

    def draw(this):

        super().draw()
        if this.getColor() == white:
            display.blit(this.__whiteImg, (this.getPos().x + 2, this.getPos().y
+ 2))
        else:
            display.blit(this.__yellowImg, (this.getPos().x + 2,
this.getPos().y + 2))

    def setPermColor(this, color):

        this.__permColor = color

    def resetColor(this):

        this.setColor(this.__permColor)

class Settings:

    def __init__(this, saveInterval, rotationSteps):

        this.__saveInterval = saveInterval
```

```
        this.__rotationSteps = rotationSteps
        this.__buttons = [
            Button(Pos(0, 0), (20, 40), "set_saveInterval"),
            Button(Pos(0, 0), (20, 40), "set_rotationSteps")
        ]

    def getsaveInterval(this):

        return this.__saveInterval

    def getRotationSteps(this):

        return this.__rotationSteps

    def getButton(this, index):

        return this.__buttons[index]

    def setsaveInterval(this, saveInterval):

        this.__saveInterval = saveInterval

    def setRotationSteps(this, rotationSteps):

        this.__rotationSteps = rotationSteps

# Program settings

windowSize = [1094, 671]
smallFontSize = 24
fontSize = 48
bigFontSize = 64
titleFontSize = 100
darkgrey  = ( 32,  32,  32)
lightgrey = (128, 128, 128)
white     = (255, 255, 255)
yellow    = (245, 237,  85)
green     = ( 78, 211,  78)

globalSettings = Settings(5, 100)
shapeList = ["Circle", "Triangle", "Rectangle", "Pentagon", "Hexagon"]
shapeTypes = ["Hollow_", "Partial_", "Filled_"]
assetFolder = "assets\\"
dbFileName = assetFolder + "weights.db"
shapeRes = 256 # Sets the resolution of the shapes that get randomly generated
weightRes = 64 # Sets the resolution of the weights for the shapes

pyg.display.set_caption("FigurAI")
display = pyg.display.set_mode(windowSize)
smallFont = pyg.font.Font(assetFolder + "AsapFont.ttf", smallFontSize)
font = pyg.font.Font(assetFolder + "BebasNeueFont.ttf", fontSize)
bigFont = pyg.font.Font(assetFolder + "BebasNeueFont.ttf", bigFontSize)
titleFont = pyg.font.Font(assetFolder + "StretchProFont.otf", titleFontSize)
```

bitmap.py:

```
import math
from init import weightRes, Pos, numpy

# bitmap functions

def copyBitmap(bitmapToCopy):

    bitmapRes = len(bitmapToCopy)

    bitmap = numpy.empty((bitmapRes, bitmapRes), dtype=type(bitmapToCopy))
    for y in range(bitmapRes):
        for x in range(bitmapRes):
            bitmap[x, y] = bitmapToCopy[x, y]

    return bitmap

def cropBitmap(bitmap):

    bitmapRes = len(bitmap)
    scaleFactor = weightRes / bitmapRes

    croppedBitmap = numpy.full((weightRes, weightRes), 0, dtype=bool)
    corners = [Pos(bitmapRes - 1, bitmapRes - 1), Pos(0, 0)]
    for y in range(bitmapRes):
        for x in range(bitmapRes):
            if bitmap[x, y]:

                if x < corners[0].x:
                    corners[0].x = x

                if x > corners[1].x:
                    corners[1].x = x

                if y < corners[0].y:
                    corners[0].y = y

                if y > corners[1].y:
                    corners[1].y = y

    width = corners[1].x - corners[0].x
    height = corners[1].y - corners[0].y
    center = Pos(width / 2 + corners[0].x, height / 2 + corners[0].y)
    xStretchScale = 1
    yStretchScale = 1
    if width >= height:
        frame = width
        yStretchScale = 2 * (center.y - corners[0].y) / frame
        corners[0].y -= (width - height) / 2
    else:
        frame = height
        xStretchScale = 2 * (center.x - corners[0].x) / frame
        corners[0].x -= (height - width) / 2

    multiplier = frame / (bitmapRes - 1)
    for y in range(bitmapRes):
        for x in range(bitmapRes):
```

```
            xIndex = int(center.x + (x * multiplier - frame / 2) *
xStretchScale)
            yIndex = int(center.y + (y * multiplier - frame / 2) *
yStretchScale)
            if xIndex < bitmapRes and yIndex < bitmapRes:
                if bitmap[xIndex, yIndex]:
                    croppedBitmap[int(x * scaleFactor), int(y * scaleFactor)] =
True

    return croppedBitmap

def rotateBitmap(bitmap, degrees):

    bitmapRes = len(bitmap)

    flipCount = 0
    while degrees >= 90:
        degrees -= 90
        flipCount += 1
        if flipCount > 3:
            flipCount = 0

    for flip in range(flipCount):

        rotatedBitmap = numpy.full((bitmapRes, bitmapRes), False, dtype=bool)
        for y in range(bitmapRes):
            for x in range(bitmapRes):
                if bitmap[x, y]:
                    rotatedBitmap[bitmapRes - y - 1, x] = True

        bitmap = copyBitmap(rotatedBitmap)

    if degrees != 0:

        rotatedRes = int(bitmapRes * math.sqrt(2))
        rotatedBitmap = numpy.full((rotatedRes, rotatedRes), False, dtype=bool)
        center = rotatedRes / 2
        sinAngle = math.sin(math.radians(degrees))
        cosAngle = math.cos(math.radians(degrees))

        for y in range(rotatedRes):
            for x in range(rotatedRes):
                xIndex = int((x - center) * cosAngle + (y - center) * sinAngle
+ bitmapRes / 2)
                yIndex = int((x - center) * -sinAngle + (y - center) * cosAngle
+ bitmapRes / 2)
                if xIndex in range(bitmapRes) and yIndex in range(bitmapRes)
and bitmap[xIndex, yIndex]:
                    rotatedBitmap[x, y] = True

        return rotatedBitmap

    else:
        return bitmap

def validateBitmap(bitmap):

    bitmapRes = len(bitmap)

    check = False
```

```
    for y in range(bitmapRes):
        for x in range(bitmapRes):
            if not bitmap[x, y]:
                check = True

    return check
```

dbConn.py:

```
import sqlite3 as sql
from init import dbFileName, weightRes, numpy

# Database functions

def connectToDB():

    conn = None
    try:
        conn = sql.connect(dbFileName)
    except sql.Error as e:
        print(e)

    if conn is None:
        print("Error! Cannot create a connection to '" + dbFileName + "'")

    return conn

def readWeights(tableName):

    weightmap = numpy.full((weightRes, weightRes), 0, dtype=int)
    dbConn = connectToDB()
    if dbConn is not None:

        cur = dbConn.cursor()
        cur.execute("CREATE TABLE IF NOT EXISTS " + tableName + " (xIndex INT
NOT NULL, yIndex INT NOT NULL, weightValue INT NOT NULL);")
        cur.execute("SELECT * FROM " + tableName)
        results = cur.fetchall()

        if len(results) == weightRes ** 2:
            for row in results:
                weightmap[row[0], row[1]] = row[2]

        else:
            if len(results) == 0:
                print("Error! '" + dbFileName + "' Unable to find " + tableName
+ " weights! Creating new weights...")
            else:
                print("Error! '" + dbFileName + "' " + tableName + " weights
are not formatted to " + str(weightRes) + " x " + str(weightRes) + "! Resetting
weights...")

            cur.execute("DELETE FROM " + tableName)
            for y in range(weightRes):
                for x in range(weightRes):
                    cur.execute("INSERT INTO " + tableName + " VALUES (?, ?,
?)", (x, y, 0))
            dbConn.commit()
            print("Successfully created new " + tableName + " weights\n")

        cur.close()
        dbConn.close()

    return weightmap

def updateDB(tableName, weightmap):
```

```
    print("\nSaving " + tableName + " weights...")
    dbConn = connectToDB()
    if dbConn is not None:

        cur = dbConn.cursor()
        cur.execute("DELETE FROM " + tableName)
        for y in range(weightRes):
            for x in range(weightRes):
                cur.execute("INSERT INTO " + tableName + " VALUES (?, ?, ?)",
(x, y, int(weightmap[x, y])))

        dbConn.commit()
        cur.close()
        dbConn.close()
        print("Successfully saved " + tableName + " weights")
```

shapeGen.py:

```
import math
from random import randint, uniform as randfloat
from init import shapeRes
from bitmap import *

# Random shape generation functions

def randAttributes():

    randomness = randfloat(0, 1)
    center = Pos(randint(int(shapeRes / 3), int(shapeRes * 2 / 3)),
randint(int(shapeRes / 3), int(shapeRes * 2 / 3)))
    radius = int((randomness + 1) * shapeRes / 6)
    heightMultiplier = randfloat(0.3, 1)
    widthMultiplier = randfloat(0.3, 1)
    brushSize = int(randfloat(0, 7 * randomness)) + 2

    return randomness, center, radius, heightMultiplier, widthMultiplier,
brushSize

def fillShape(bitmap, startPos, volume):

    bitmapRes = len(bitmap)
    bitmapCopy = copyBitmap(bitmap)

    bitmapCopy[startPos.x, startPos.y] = True
    pointsToCheck = [startPos]
    for pos in pointsToCheck:
        if randfloat(0, 1) <= volume:
            bitmap[pos.x, pos.y] = True

        if pos.x - 1 >= 0 and not bitmapCopy[pos.x - 1, pos.y]:
            bitmapCopy[pos.x - 1, pos.y] = True
            pointsToCheck.append(Pos(pos.x - 1, pos.y))

        if pos.y + 1 < bitmapRes and not bitmapCopy[pos.x, pos.y + 1]:
            bitmapCopy[pos.x, pos.y + 1] = True
            pointsToCheck.append(Pos(pos.x, pos.y + 1))

        if pos.x + 1 < bitmapRes and not bitmapCopy[pos.x + 1, pos.y]:
            bitmapCopy[pos.x + 1, pos.y] = True
            pointsToCheck.append(Pos(pos.x + 1, pos.y))

        if pos.y - 1 >= 0 and not bitmapCopy[pos.x, pos.y - 1]:
            bitmapCopy[pos.x, pos.y - 1] = True
            pointsToCheck.append(Pos(pos.x, pos.y - 1))

    return bitmap

def drawDot(bitmap, bitmapPos, brushSize, eraser = False):

    bitmapRes = len(bitmap)

    for y in range(bitmapPos.y - brushSize, bitmapPos.y + brushSize + 1):
        for x in range(bitmapPos.x - brushSize, bitmapPos.x + brushSize + 1):
            if ((x - bitmapPos.x) ** 2) + ((y - bitmapPos.y) ** 2) - (brushSize
** 2) < 1 and x in range(bitmapRes) and y in range(bitmapRes):
```

```
                if not eraser and not bitmap[x, y]:
                    bitmap[x, y] = True
                if eraser and bitmap[x, y]:
                    bitmap[x, y] = False

    return bitmap

def drawLine(bitmap, pos1, pos2, brushSize, shapeType, randomness):

    if pos2.x <= pos1.x:
        temp = pos1
        pos1 = pos2
        pos2 = temp

    xDist = (pos1.x - pos2.x)
    yDist = (pos1.y - pos2.y)
    lineLength = math.sqrt(xDist ** 2 + yDist ** 2)
    if xDist != 0:
        angle = math.atan(yDist / xDist)
    else:
        if pos2.y > pos1. y:
            angle = math.pi / 2
        else:
            angle = 3 * math.pi / 2

    xMultiplier = randint(1, 3) * randomness
    xMultiplier2 = randint(3, 5) * randomness

    if shapeType == "Hollow_":
        xOffset = randfloat(0, math.tau)
        xOffset2 = randfloat(0, math.tau)
        stepNo = randfloat(lineLength * 0.1, lineLength)
        minBranch = -randomness * lineLength / 20
    else:
        xOffset = math.pi * randint(0, 1)
        xOffset2 = math.pi * randint(0, 1)
        stepNo = lineLength
        minBranch = lineLength / 20

    xOffset3 = randfloat(0, math.pi)

    yMultiplier = randfloat(1, 1 + randomness * 2) * randomness
    yMultiplier2 = randfloat(randomness * -3, 0) * randomness

    for step in range(int((randfloat(randfloat(-lineLength / 10, 0), 0) -
minBranch) * randomness), int(stepNo + minBranch + randfloat(0, randfloat(0,
randomness * lineLength / 10)) * randomness)):
        lineX = step * lineLength / stepNo
        lineDist = lineX / lineLength
        lineY = (yMultiplier * math.sin(xMultiplier * math.pi * lineDist +
xOffset) + yMultiplier2 * math.sin(xMultiplier2 * math.pi * lineDist +
xOffset2)) * math.sin(math.pi * lineDist + xOffset3)
        bitmapPos = Pos(pos1.x + int((lineX * math.cos(angle)) - (lineY *
math.sin(angle))), pos1.y + int((lineX * math.sin(angle)) + (lineY *
math.cos(angle))))
        drawDot(bitmap, bitmapPos, brushSize)

    return bitmap

def genCircle(shapeType):
```

```
    bitmap = numpy.full((shapeRes, shapeRes), False, dtype=bool)
    randomness, center, radius, heightMultiplier, widthMultiplier, brushSize =
randAttributes()

    circumference = math.tau * radius

    xMultiplier = randint(1, 5)
    xMultiplier2 = randint(5, 10)

    if shapeType == "Hollow_":
        xOffset = randfloat(0, math.tau)
        xOffset2 = randfloat(0, math.tau)
        stepNo = randfloat(circumference * 0.05, circumference)
        spiralScale = randfloat(-50, 50) * randomness
        minBranch = -randomness * circumference / 20
    else:
        xOffset = math.pi * randint(0, 1)
        xOffset2 = math.pi * randint(0, 1)
        stepNo = circumference
        spiralScale = 0
        minBranch = circumference / 20

    xOffset3 = randfloat(0, math.pi)

    yMultiplier = randfloat(1, 1 + randomness * 8) * randomness
    yMultiplier2 = randfloat(randomness * -4, 0) * randomness

    angleOffset = randfloat(0, 1)

    for step in range(int((randfloat(randfloat(-circumference / 10, 0), 0) -
minBranch) * randomness), int(stepNo + minBranch + randfloat(0, randfloat(0,
randomness * circumference / 10)) * randomness)):
        circleDist = step / stepNo
        circleY = (yMultiplier * math.sin(xMultiplier * math.pi * circleDist +
xOffset) + yMultiplier2 * math.sin(xMultiplier2 * math.pi * circleDist +
xOffset2)) * math.sin(math.pi * circleDist + xOffset3) + spiralScale *
circleDist
        angle = math.tau * (angleOffset - circleDist)
        bitmapPos = Pos(center.x + int(widthMultiplier * math.sin(angle) *
(radius - circleY)), center.y + int(heightMultiplier * math.cos(angle) *
(radius + circleY)))
        drawDot(bitmap, bitmapPos, brushSize)

    if shapeType == "Partial_":
        fillShape(bitmap, center, randfloat(0.01, 0.1))
    elif shapeType == "Filled_":
        fillShape(bitmap, center, 1)

    return bitmap

def genPolygon(shapeType, cornerNo):

    bitmap = numpy.full((shapeRes, shapeRes), False, dtype=bool)
    randomness, center, radius, heightMultiplier, widthMultiplier, brushSize =
randAttributes()

    corners = []
    for loop in range(cornerNo):
        randOffset = randfloat(-randomness / cornerNo, randomness / cornerNo)
```

```
        angle = math.tau * ((2 * loop + 1) / (2 * cornerNo)) + randOffset
        corners.append(Pos(center.x + int(widthMultiplier * (radius *
math.sin(angle) + randfloat(-1, 1) * randomness * shapeRes / 32)), center.y +
int(heightMultiplier * (radius * math.cos(angle) + randfloat(-1, 1) *
randomness * shapeRes / 32))))

    for i in range(1, len(corners)):
        drawLine(bitmap, corners[i-1], corners[i], brushSize, shapeType,
randomness)
    drawLine(bitmap, corners[0], corners[i], brushSize, shapeType, randomness)

    if shapeType == "Partial_":
        fillShape(bitmap, center, randfloat(0.01, 0.1))
    elif shapeType == "Filled_":
        fillShape(bitmap, center, 1)

    return bitmap
```